



V-LRN

VIRTUAL LEARNING NETWORK

Discrete Mathematics

Block – 1

Unit - 1

Propositional Calculus

Virtual Learning Network



Topics to be Covered

- Introduction
- Propositions
- Logical Connectives
 - Disjunction
 - Conjunction
 - Negation
 - Conditional Connectives
 - Precedence Rule
- Logical Equivalence
- Logical Quantifiers



Introduction

- In the introduction to the block you have read about what symbolic logic is.
- Using it we can formalise our arguments and logical reasoning in a manner that can easily show if the reasoning is valid, or is a fallacy.
- How we symbolise the reasoning is what is presented in this unit.
- You will also see that a statement can either be true or false.
- Accordingly, as you will see, we will give the statement a truth value T or F.



Propositions

- A proposition is a collection of declarative statements that has either a truth value "true" or a truth value "false".
- A propositional consists of propositional variables and connectives.
- We denote the propositional variables by capital letters (A, B, etc).
- The connectives connect the propositional variables.
- Some examples of Propositions are given below
 - "Man is Mortal", it returns truth value "TRUE"
 - " $12 + 9 = 3 - 2$ ", it returns truth value "FALSE"

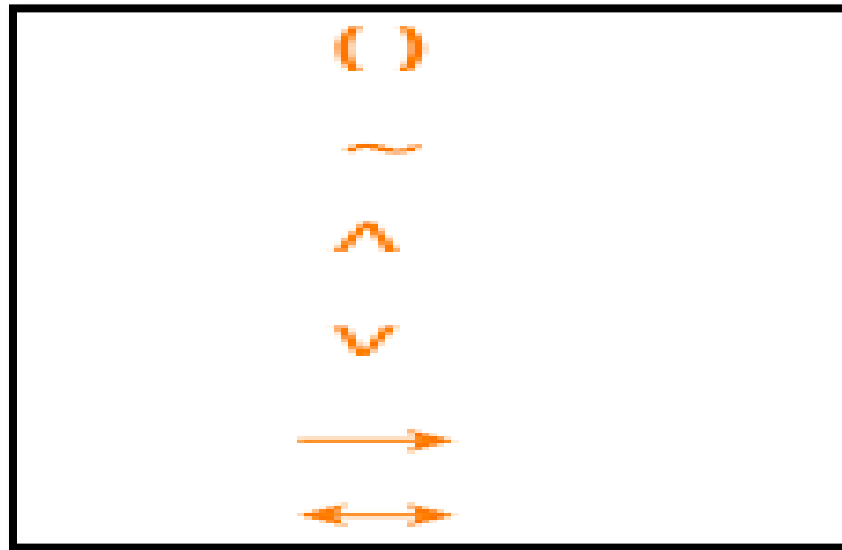


Propositions(Contd....)

- The following is not a Proposition –
 - "A is less than 2". It is because unless we give a specific value of A, we cannot say whether the statement is true or false.
- Sometimes, as in the context of logic circuits we will use 1 instead of T and 0 instead of F.

Logical Connectives

- Statements in mathematical logic are combinations of simpler statements joined by words and phrases like 'and', 'or', 'if ... then'. 'if and only if', etc
- These words and phrases are called logical connectives.



Logical Connectives(Contd....)

- **Disjunction :**

- The disjunction of two propositions p and q is the compound statement p or q .
- Denoted by $p \vee q$.

Truth table for disjunction

p	q	$p \vee q$
T	T	T
T	F	T
F	T	T
F	F	F

Logical Connectives(Contd....)

– Example

- Obtain the truth value of the disjunction of 'The earth is flat'. and ' $3 + 5 = 2$ '.

– Solution

- Let p denote 'The earth is flat,' and q denote ' $3 + 5 = 2$ '. Then we know that the truth values of both p and q are F. Therefore, the truth value of $p \vee q$ is F.

Logical Connectives(Contd....)

- **Exclusive Disjunction:**

- Two propositions p and q is the statement 'Either p is true or q is true, but both are not true.'
- Either p is true or q is true, but both are not true.'
- We denote this by $p \oplus q$.
- Symbol
 - $\oplus$

Logical Connectives(Contd....)

- **Conjunction:**
 - The compound statement 'p and q' the conjunction of the statements p and q.
 - We denote this by $p \wedge q$.

Truth table for conjunction

P	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F



Logical Connectives(Contd....)

– Example:

– Obtain the truth value of the conjunction of '2 ÷ 5 = 1' and 'Padma is in Bangalore.'

– Solution

- Let $p : 2 \div 5 = 1$, and q : Padma is in Bangalore.
- Then the truth value of p is F.
- According to the conjunction table the truth value of $p \wedge q$ is F.

Logical Connectives(Contd....)

- **Negation :**

- The negation of a proposition p is 'not p '.
- Denoted by $\sim p$
- For example:
 - If p is 'Dolly is at the study center.',
 - then $\sim p$ is 'Dolly is not at the study center'.
- **Symbol**
 - $\sim$



Logical Connectives(Contd....)

- **Conditional Connectives :**

- Given any two propositions p and q , we denote the statement 'If p , then q ' by $p \rightarrow q$.
- We also read this as ' p implies q '. or ' p is sufficient for q ', or ' p only if q '.
- We also call p the hypothesis and q the conclusion
- $p \rightarrow q$ is called a conditional statement or a conditional proposition

Logical Connectives(Contd....)

• Truth table for implication

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

– The converse of $p \rightarrow q$ is $q \rightarrow p$. In this case we also say 'p is necessary for q', or 'p if q

– **Symbol**

• $\rightarrow$

Logical Connectives(Contd....)

- The two connectives $\rightarrow$ and $\leftrightarrow$ are called conditional connectives.
- The compound statement $(p \rightarrow q) \wedge (q \rightarrow p)$ is the biconditional of p and q .
- We denote it by $p \leftrightarrow q$, and read it as 'p if and only q'.

Table : Truth table for two-way implication.

p	q	$p \rightarrow q$	$q \rightarrow p$	$p \leftrightarrow q$
T	T	T	T	T
T	F	F	T	F
F	T	T	F	F
F	F	T	T	T

Logical Connectives(Contd....)

- **Precedence Rule:**

Operator	Precedence
$\neg$	1
$\wedge$	2
$\vee$	3
$\rightarrow$	4
$\leftrightarrow$	5

Logical Connectives(Contd....)

- **Example:** Write down the truth table of $p \rightarrow q \wedge \sim r$
 $\leftrightarrow r \oplus q$

p	q	r	$\sim r$	$q \wedge \sim r$	$r \oplus q$	$p \rightarrow q \wedge \sim r$	$p \rightarrow q \wedge \sim r \leftrightarrow r \oplus q$
T	T	T	F	F	F	F	T
T	T	F	T	T	T	T	T
T	F	T	F	F	T	F	F
T	F	F	T	F	F	F	T
F	T	T	F	F	F	T	F
F	T	F	T	T	T	T	T
F	F	T	F	F	T	T	T
F	F	F	T	F	F	T	F

Logical Equivalence

- Two propositions r and s logically equivalent provided they have the same truth value for every choice of truth values of simple propositions involved in them.
- We denote this fact by $r \equiv s$.
- **Example:**
 - For any two propositions p and q , show that $\sim (p \vee q) \equiv \sim p \wedge \sim q$.

p	q	$\sim p$	$\sim q$	$p \vee q$	$\sim (p \vee q)$	$\sim p \wedge \sim q$
T	T	F	F	T	F	F
T	F	F	T	T	F	F
F	T	T	F	T	F	F
F	F	T	T	F	T	T

Logical Equivalence(Contd....)

- **De Morgan's Laws**

- **Double negation law** : $\sim (\sim p) \equiv p$

- **Idempotent laws**: $p \wedge p \equiv p$,
 $p \vee p \equiv p$

- **Commutativity**: $p \vee q \equiv q \vee p$
 $p \wedge q \equiv q \wedge p$

- **Associativity**: $(p \vee q) \vee r \equiv p \vee (q \vee r)$
 $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$

- **Distributivity**: $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$
 $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$

Logical Equivalence(Contd....)

- **Tautology:** A compound proposition that is true for all possible truth values of the simple propositions involved in it is called a tautology.
- **Contradiction:** A proposition that is false for all possible truth values of the simple propositions that constitute it is called a contradiction.
- **Example:** Verify that $p \wedge q \wedge \sim p$ is a contradiction and $p \rightarrow q \leftrightarrow \sim p \vee q$ is a tautology

p	q	$\sim p$	$p \wedge q$	$p \wedge q \wedge \sim p$	$p \rightarrow q$	$\sim p \vee q$	$p \rightarrow q \leftrightarrow \sim p \vee q$
T	T	F	T	F	T	T	T
T	F	F	F	F	F	F	T
F	T	T	F	F	T	T	T
F	F	T	F	F	T	T	T

Logical Equivalence(Contd....)

- **Example :**
 - Show that $(p \rightarrow \sim q) \wedge (p \rightarrow \sim r) \equiv \sim [p \wedge (q \vee r)]$.

- **Solution :**

$$\begin{aligned} & (p \rightarrow \sim q) \wedge (p \rightarrow \sim r) \\ & (\sim p \vee q) \wedge (\sim p \vee \sim r), \\ & \sim p \vee (\sim q \wedge \sim r), \text{ by distributivity} \\ & \sim p \vee [\sim (q \vee r)], \text{ by De Morgan's laws} \\ & \sim [p \wedge (q \vee r)], \text{ by De Morgan's laws} \end{aligned}$$

Logical Quantifiers

- A propositional function, or a predicate, in a variable x is a sentence $p(x)$ involving x that becomes a proposition when we give x a definite value from the set of values it can take.
- $\exists$ is called the existential quantifier.
- We use the symbol ' $\exists$ ' to denote this quantifier, 'there exists'.
- $\forall$ is called the universal quantifier.
- Mathematical symbol for the quantifier 'for all', which is ' $\forall$ '.

Logical Quantifiers(Contd....)

- **Example 1:**
 - ‘There is a criminal who has committed every crime.’
- **Solution:**
 - We could write this in symbols as $(\exists c \in A) (\forall x \in B) (c \text{ has committed } x)$.
 - Where, of course, A is the set of criminals and B is the set of crimes (determined by law).

Summary

- In this unit, we have considered the following points.
 - What a mathematically acceptable statement (or proposition) is.
 - The definition and use of logical connectives
 - The truth tables corresponding to the 6 logical connectives.
 - Rule of precedence
 - The meaning and use of logical equivalence, denoted by ' $\equiv$ '

Thank
you



V-LRN

VIRTUAL LEARNING NETWORK

Discrete Mathematics

Block – 1

Unit - 2

Methods of Proof

Virtual Learning Network



Topics to be Covered

- Introduction
- What is a Proof?
- Different Methods of Proof
 - Direct Proof
 - Indirect Proofs
 - Counterexamples
- Principle of Induction



Introduction

- In this unit, we shall discuss ways in which statements can be linked to form a logically valid argument.
- Throughout your mathematical studies you would have come across the terms 'theorem' and 'proof'.
- We shall talk about what a theorem is and what constitutes a mathematically acceptable proof.
- These are the different methods used for proving or disproving a statement.
- As you go through the different types of valid arguments.

What is a Proof?

- An argument (in mathematics or logic) is a finite sequence of statements $p_1, \dots, p_n, p$ such that $(p_1 \wedge p_2 \wedge \dots \wedge p_n) \rightarrow p$.
- Each statement in the sequence, $p_1, p_2, \dots, p_n$ is called a premise (or an assumption, or a hypothesis). The final statement p is called the conclusion.
- **Example:**
 - Give an argument to show that the mathematical statement 'For any two sets A and B , $A \cap B \subseteq A$ ' is true.

What is a Proof?(Contd....)

- **Solution:**

- One argument could be the following.
- Let x be an arbitrary element of $A \cap B$.
- Then $x \in A$ and $x \in B$, by definition of ' $\cap$ '.
- Therefore, $x \in A$. This is true for every x in $A \cap B$.
- Therefore, $A \cap B \subseteq A$, by definition of ' $\subseteq$ '.





What is a Proof? (Contd....)

- We say that a proposition p follows logically from propositions p_1 , p_2 , $\cdot \cdot \cdot$, p_n if p must be true whenever p_1 , p_2 , $\cdot \cdot \cdot$, p_n are true, i.e.,
- $(p_1 \wedge p_2 \wedge \cdot \cdot \cdot \wedge p_n) \Rightarrow p$.
- For any two propositions r and s , ' $r \Rightarrow s$ ' denotes 's is true whenever r is true.'
- We say that a proposition p follows logically from propositions p_1 , p_2 , $\cdot \cdot \cdot$, p_n if p must be true whenever p_1 , p_2 , $\cdot \cdot \cdot$, p_n are true, i.e.,
- $(p_1 \wedge p_2 \wedge \cdot \cdot \cdot \wedge p_n) \Rightarrow p$. [Here, note the use of the implication arrow ' $\Rightarrow$ '. For any two propositions r and s , ' $r \Rightarrow s$ ' denotes 's is true whenever r is true.'

What is a Proof? (Contd....)

- **Example:**

- If Maya sees the movie, she won't finish her homework.

Maya won't finish her homework.

Therefore, Maya sees the movie.

- **Solution**

- The given argument is of the form $[(p \rightarrow q) \wedge q] \Rightarrow p$

p	q	$p \rightarrow q$	$(p \rightarrow q) \wedge q$
T	T	T	T
T	F	F	F
F	T	T	T
F	F	T	F

What is a Proof? (Contd....)

- This last column gives the truth values of the premises.
- The first column given corresponding truth values of the conclusion.
- Now, the argument will only be valid if whenever both the premises are true, the conclusion is true.
- This happens in the first row, but not in the third row. Therefore, the argument is not valid.

Different Methods of Proof

- **Direct Proof :**
- A direct proof of $p \Rightarrow q$ is a logically valid argument that begins with the assumptions that p is true and, in one or more applications of the law of detachment, concludes that q must be true
- So, to construct a direct proof of $p \Rightarrow q$, we start by assuming that p is true.
- Then, in one or more steps of the form $p \Rightarrow q_1$, $q_1 \Rightarrow q_2$,, $q_n \Rightarrow q$.
- we conclude that q is true



Different Methods of Proof(Contd....)

- **Example 1:** Give a direct proof of the statement 'The product of two odd integers is odd'.
- **Solution:** Let two odd integers x and y .
 - So our hypothesis is p : x and y
 - The conclusion we want to reach is q : xy is odd.
 - Let us first prove that $p \Rightarrow q$.
 - Since x is odd, $x = 2m + 1$ for some integer m . Similarly, $y = 2n + 1$ for some integer n .
 - Then $xy = (2m + 1)(2n + 1) = 2(2mn + m + n) + 1$
Therefore, xy is odd
 - So we have shown that $p \Rightarrow q$.

Different Methods of Proof(Contd....)

- **Example 2:** Give a direct proof of the theorem ‘The square of an even integer is an even integer.’
- **Solution:** Let x be an even number (i.e., we assume $p(x)$ is true).
 - Then $x = 2n$, for some integer n (we apply the definition of an even number).
 - Then $x^2 = (2n)^2 = 4n^2 = 2(2n^2)$.
 - x^2 is even (i.e., $q(x)$ is true).



Different Methods of Proof(Contd....)

- **Indirect Proofs:**

- **Proof by contrapositive:** In the first method, we use the fact that the proposition $p \Rightarrow q$ is logically equivalent to its contrapositive $(\sim q \Rightarrow \sim p)$,
- $(p \Rightarrow q) \equiv (\sim q \Rightarrow \sim p)$
- what we do to prove $p \Rightarrow q$
- In this method is to assume that q is false and then show that p is false.



Different Methods of Proof(Contd....)

- **Example:** Prove that 'If $x, y \in \mathbb{Z}$ such that xy is odd, then both x and y are odd.', by proving its contrapositive.
- **Solution:** Let us name the statements involved as below.
 - p : xy is odd q : both x and y are odd.
 - So, $\sim p$: xy is even, and $\sim q$: x is even or y is even, or both are even.
 - We want to prove $p \Rightarrow q$, by proving that $\sim q \Rightarrow \sim p$. So we start by assuming that $\sim q$ is true, i.e., we suppose that x is even.
 - The $x = 2n$ for some $n \in \mathbb{N}$. Therefore, $xy = 2ny$. Therefore xy is even, by definition. That is, $\sim p$ is true.



Different Methods of Proof(Contd....)

- **Proof by contradiction:**
 - In this method, to prove q is true, we start by assuming that q is false (i.e., $\sim q$ is true).
 - Then, by a logical argument we arrive at a situation where a statement is true as well as false, i.e.,
 - We reach a contradiction $r \wedge \sim r$ for some statement that is always false.
 - This can only happen when $\sim q$ is false also. Therefore, q must be true.



Different Methods of Proof(Contd....)

- **Example:** Show that 5 is irrational.
- **Solution:** Let us try and prove the given statement by contradiction.
 - For this, we begin by assuming that 5 is rational.
 - This means that there exist positive integers a and b such that $5 = \frac{a}{b}$, where a and b have no common factors.
 - This implies $a = 5b \Rightarrow a^2 = 5b^2 \Rightarrow 5 \mid a^2 \Rightarrow 5 \mid a$.
Therefore, by definition, $a = 5c$ for some $c \in \mathbb{Z}$.
Therefore, $a^2 = 25c^2$ But $a^2 = 5b^2$ also.



Different Methods of Proof(Contd....)

- So $25c^2 = 5b^2 \Rightarrow 5c^2 = b^2 \Rightarrow 5|b^2 \Rightarrow 5|b$.
- But now we find that 5 divides both a and b, which contradicts our earlier assumption that a and b have no common factor.
- Therefore, we conclude that our assumption that $\sqrt{5}$ is rational is false, i.e, $\sqrt{5}$ is irrational.

Different Methods of Proof(Contd....)

- **Counter Example** to a statement p proves that p is false, i.e., $\sim p$ is true.
- **Example:**
 - Disprove the following statement:
$$(\forall a \in \mathbb{R}) (\forall b \in \mathbb{R}) [(a^2 = b^2) \Rightarrow (a = b)].$$
- **Solution:**
 - A good way of disproving it is to look for a counterexample, that is, a pair of real numbers a and b for which $a^2 = b^2$

Principle of Induction

- **Example:** : Use mathematical induction to prove that $1^2 + 2^2 + 3^2 + \dots + n^2 = n/6 (n + 1) (2n + 1) \forall n \in \mathbb{N}$.
- **Solution:** We call $p(n)$ the predicate $1^2 + 2^2 + 3^2 + \dots + n^2 = 6 n (n + 1) (2n + 1)$. Since we want to prove it for every $n \in \mathbb{N}$, we take $m = 1$.
 - **Step 1:** $p(1)$ is $1^2 = 1/6(1 + 1) (2 + 1)$, which is true
 - **Step 2:** Suppose for an arbitrary $k \in \mathbb{N}$, $p(k)$ is true, i.e., $1^2 + 2^2 + \dots + k^2 = 6 k (k + 1) (2k + 1)$ is true.
 - **Step 3:** To check if the assumption in step 2 implies that $p(k + 1)$ is true. Let's see.

Principle of Induction(Contd....)

- **Example:** : Use mathematical induction to prove that $1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6} \forall n \in \mathbb{N}$.
- **Solution:** We call $p(n)$ the predicate $1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$. Since we want to prove it for every $n \in \mathbb{N}$, we take $m = 1$.
 - **Step 1:** $p(1)$ is $1^2 = \frac{1}{6}(1+1)(2+1)$, which is true
 - **Step 2:** Suppose for an arbitrary $k \in \mathbb{N}$, $p(k)$ is true, i.e., $1^2 + 2^2 + \dots + k^2 = \frac{k(k+1)(2k+1)}{6}$ is true.
 - **Step 3:** To check if the assumption in step 2 implies that $p(k+1)$ is true. Let's see.

Principle of Induction(Contd....)

- $P(k + 1)$ is $1^2 + 2^2 + \dots + k^2 + (k + 1)^2 = (k + 1)/6 (k + 2) (2k + 3)$
- $(1^2 + 2^2 + \dots + k^2) + (k + 1)^2 = (k + 1)/6 (k + 2) (2k + 3)$
- $k/6 (k + 1) (2k + 1) + (k + 1)^2$ since $p(k)$ is true.
- $(k + 1)/6 [k(2k + 1) + 6(k + 1)]$
- $(k + 1)/6 [(2k^2 + 7k + 6)]$
- $(k + 1)/6 [(k + 2) (2k + 3)]$ which is true.
- So, $p(k)$ is true implies that $p(k + 1)$ is true.

Summary

- In this unit, you have studied the following points.
 - What constitutes a proof of a mathematical statement, including 4 commonly used rules of inference.
 - The description and examples of a direct proof, which is based on modus ponens
 - Two types of indirect proofs : proof by contrapositive and proof by contradiction
 - The use of counterexamples for disproving a statement
 - The “strong” and “weak” forms of the principle of mathematical induction.

Thank
You



V-LRN

VIRTUAL LEARNING NETWORK

Discrete Mathematics

Block – 1

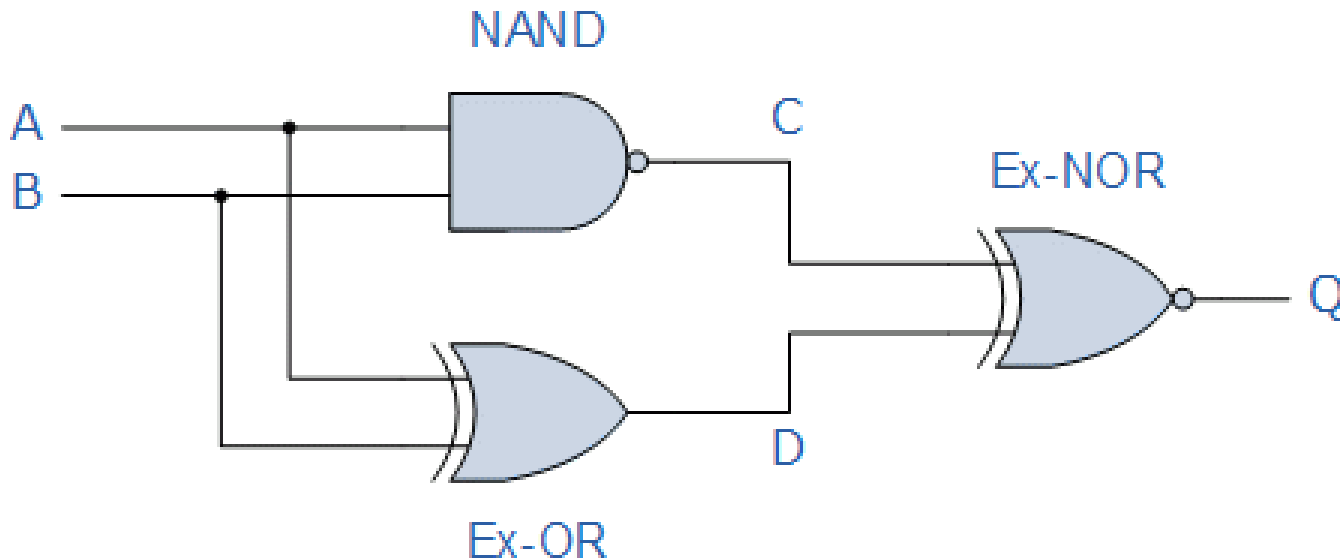
Unit - 3

Boolean Algebra and Circuits

Virtual Learning Network

Topics to be Covered

- Introduction
- Boolean Algebras
- Logic Circuits
- Boolean Functions





Introduction

- A technique based on this analogy to express and manipulate simple switching circuits algebraically.
- In this unit, we shall discuss the symbolic logic techniques which are required for the algebraic understanding of circuits and computer logic.
- We shall introduce you to Boolean algebras with the help of certain examples based on objects you are already familiar with.
- Boolean expressions and logic circuits.
- The overall functioning of a circuit mathematically in terms of certain suitably defined functions called Boolean functions.

Boolean Algebras

- A Boolean algebra B is an algebraic structure which consists of a set X ($\neq \emptyset$) having two binary operations (denoted by $\vee$ and $\wedge$), one unary operation (denoted by $'$) and two specially defined elements 0 and 1 (say), which satisfy the following five laws for all $x, y, z \in X$.

- **Associative Laws:**

$$x \vee (y \vee z) = (x \vee y) \vee z,$$

$$x \wedge (y \wedge z) = (x \wedge y) \wedge z$$

- **Identity Laws:**

$$x \vee 0 = x,$$

$$x \wedge 1 = x$$

Boolean Algebras(Contd....)

– Commutative Laws:

$$x \vee y = y \vee x$$

$$x \wedge y = y \wedge x$$

– Distributive Laws:

$$x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z),$$

$$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$$

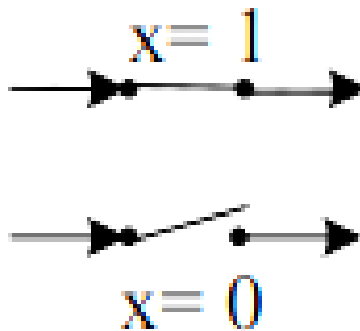
– Complementation Laws:

$$x \wedge x' = 0,$$

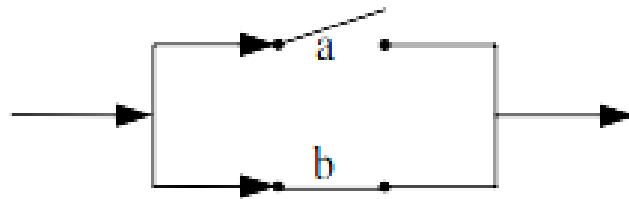
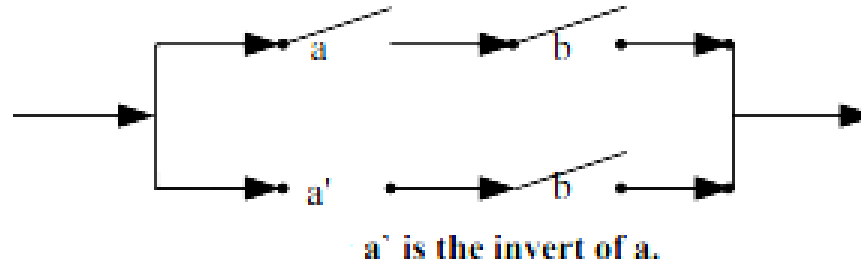
$$x \vee x' = 1$$

Boolean Algebras(Contd....)

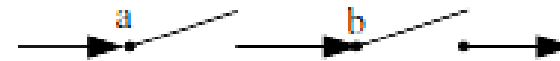
- A switch is a device which allows the current to flow only when it is placed in the ON position, i.e., when the gap is closed by a conducting rod.
- Thus, the ON position of a switch is one state of a switch, called a closed state. The other state of a switch is the open state, when it is placed in the OFF position. So, a switch has two stable states.



Boolean Algebras(Contd....)



(i) Parallel Connection



(ii) Series Connection

Boolean Algebras(Contd....)

- The state values of the connections a par b and a ser b, for different pairs of state values of switches a and b, are as given in the tables below

Table 2: State values of a par b and a ser b.

s.v. of a	s.v. of b	s.v. of a par b
0	0	0
0	1	1
1	0	1
1	1	1

s.v. of a	s.v. of b	s.v. of a ser b
0	0	0
0	1	0
1	0	0
1	1	1

Boolean Algebras(Contd....)

- **Idempotent laws** : $x \vee x = x, x \wedge x = x.$
- **Absorption laws** : $x \vee (x \wedge y) = x, x \wedge (x \vee y) = x.$
- **Involution law** : $(x')' = x.$
- **De Morgan's laws** : $(x \vee y)' = x' \wedge y', (x \wedge y)' = x' \vee y'.$

- **Example:** Reduce the Boolean expressions to a simpler form.

- $X(x_1, x_2) = (x_1 \wedge x_2) \wedge (x_1 \wedge x'_2);$
- $X(x_1, x_2, x_3) = (x_1 \wedge x_2) \vee (x_1 \wedge x_2 \wedge x_3) \vee (x_1 \wedge x_3).$

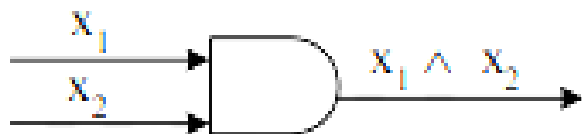
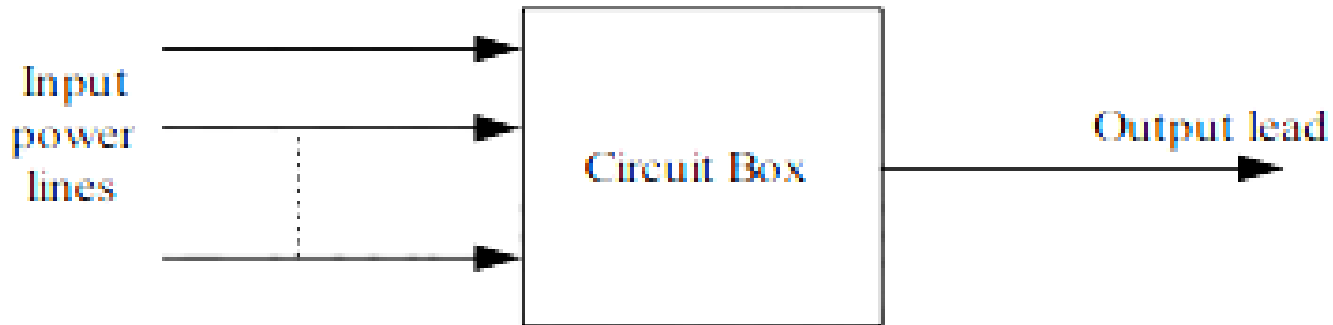
Boolean Algebras(Contd....)

- **Solution 1:** Here we can write
 - $(x_1 \wedge x_2) \wedge (x_1 \wedge x'_2) = ((x_1 \wedge x_2) \wedge x_1) \wedge x'_2$
(Associative law)
 - $(x_1 \wedge x_2) \wedge x'_2$ (Absorption law)
 - $x_1 \wedge (x_2 \wedge x'_2)$ (Associative law)
 - $x_1 \wedge 0$ (Complementation law)
 - 0 . (Identity law)
- **Solution 2:** We can write $(x_1 \wedge x_2) \vee (x_1 \wedge x'_2 \wedge x_3) \vee (x_1 \wedge x_3)$
 - $[x_1 \wedge \{x_2 \vee (x'_2 \wedge x_3)\}] \wedge (x_1 \wedge x_3)$ (Distributive law)
 - $x_1 \wedge \{(x_2 \vee x'_2) \wedge (x_2 \vee x_3)\} \wedge (x_1 \wedge x_3)$ (Distributive law)

Boolean Algebras(Contd....)

- $[x_1 \wedge \{1 \wedge (x_2 \vee x_3)\}] \wedge (x_1 \wedge x_3)$ (Complementation law)
- $[x_1 \wedge (x_2 \vee x_3)] \wedge (x_1 \wedge x_3)$ (Identity law)
- $[(x_1 \wedge x_2) \vee (x_1 \wedge x_3)] \wedge (x_1 \wedge x_3)$ (Distributive law)
- $[(x_1 \wedge x_2) \wedge (x_1 \wedge x_3)] \vee [(x_1 \wedge x_3) \wedge (x_1 \wedge x_3)]$
(Distributive law)
- $(x_1 \wedge x_2 \wedge x_3) \vee (x_1 \wedge x_3)$ (Idemp., & assoc. laws)
- $x_1 \wedge [(x_2 \wedge x_3) \vee x_3]$ (Distributive law)
- $x_1 \wedge x_3$ (Absorption law)
- Thus, the simplified form of the expression given in (b) is $(x_1 \wedge x_3)$.

Logic Circuits



Diagrammatic representation of an AND -gate

:Outputs of AND-gate

x_1	x_2	$x_1 \wedge x_2$
0	0	0
1	1	0
0	1	0
1	1	1

Logic Circuits(Contd....)

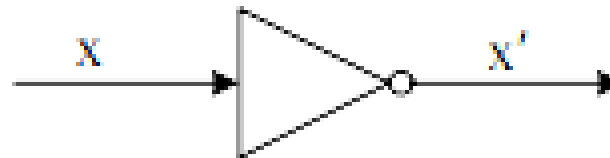


Diagrammatic representation of an OR-gate

Output of an OR-gate.

X_1	X_2	$X_1 \vee X_2$
0	0	0
0	1	1
1	0	1
1	1	1

Logic Circuits(Contd....)



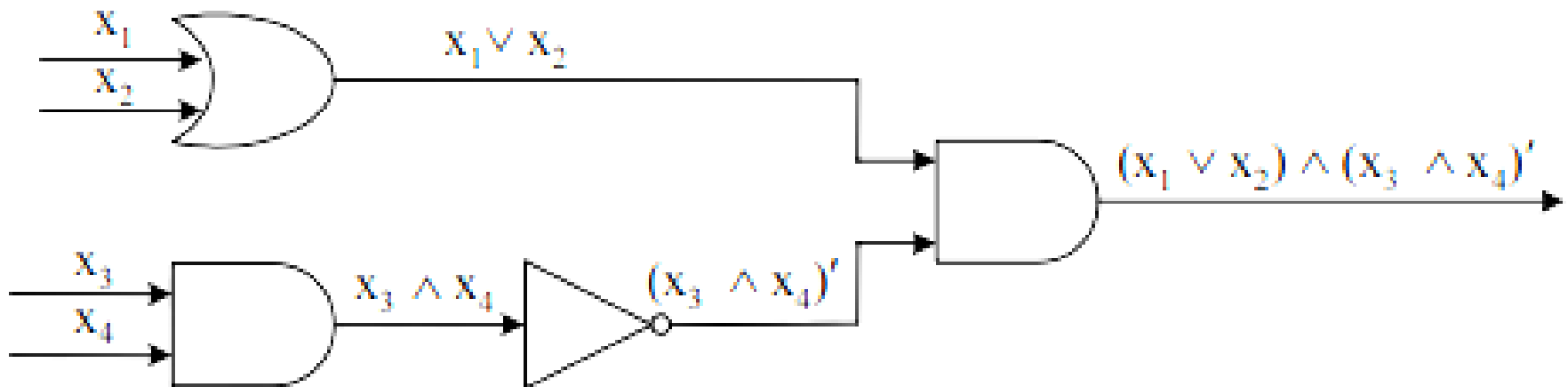
Diagrammatic representation of NOT-gate

Output of a
NOT-gate

x	x'
0	1
1	0

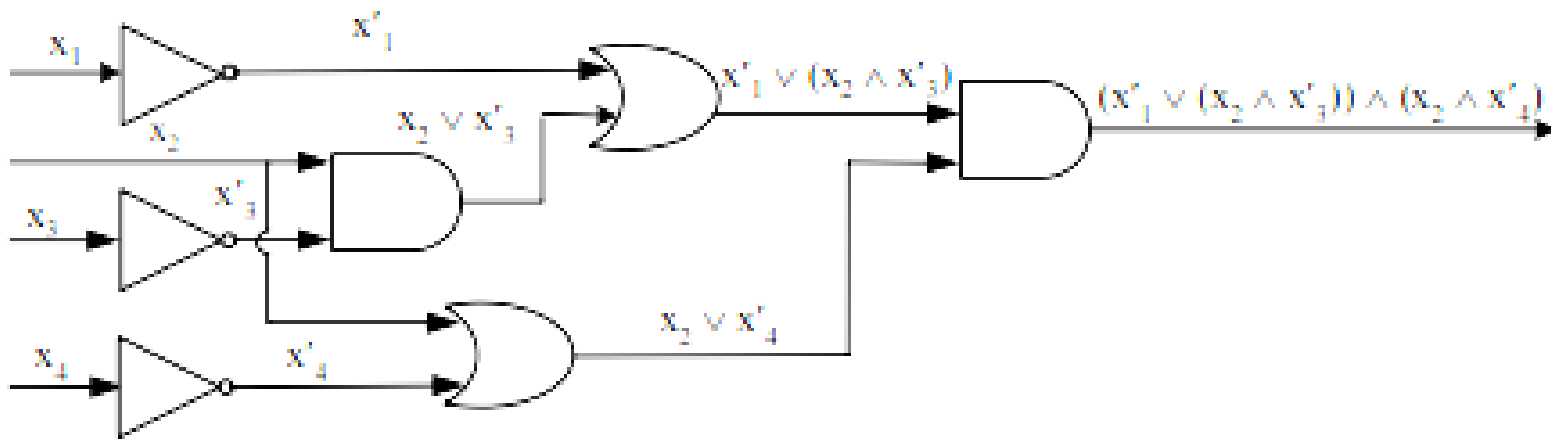
Logic Circuits(Contd....)

- Representation



Logic Circuits(Contd....)

- Expressions $(x'_1 \vee (x_2 \wedge x'_3))$ and $(x_2 \vee x'_4)$



: Circuitry for the expression $(x'_1 \vee (x_2 \wedge x'_3)) \wedge (x_2 \vee x'_4)$.

Boolean Functions

Let us first consider the Boolean expression

$$X(x_1, x_2) = x_1 \wedge x_2',$$

where x_1 and x_2 take values in $\mathcal{B} = \{0, 1\}$. You know that all the values of this expression, for different pairs of values of the variables x_1 and x_2 , can be calculated by using properties of the Boolean algebra $\mathcal{B}$. For example,

$$0 \wedge 1' = 0 \wedge 0 = 0 \Rightarrow X(0, 1) = 0.$$

Similarly, you can calculate the other values of $X(x_1, x_2) = x_1 \wedge x_2'$ over $\mathcal{B}$.

In this way we have obtained a function $f: \mathcal{B}_2 \rightarrow \mathcal{B}$, defined as follows:

$$f(e_1, e_2) = X(e_1, e_2) = e_1 \wedge e_2', \text{ where } e_1, e_2 \in \{0, 1\}.$$

So f is obtained by replacing x_i with e_i in the expression $X(x_1, x_2)$. For example, when $e_1 = 1$, $e_2 = 0$, we get $f(1, 0) = 1 \wedge 0' = 1$.

More generally, each Boolean expression $X(x_1, x_2, \dots, x_k)$ in k variables, where

Boolean Functions(Contd....)

Example 9: Let $f : \mathcal{B}^2 \rightarrow \mathcal{B}$ denote the function which is defined by the Boolean expression $X(x_1, x_2) = x'_1 \wedge x'_2$. Write the values of f in tabular form.

Solution: f is defined by $f(e_1, e_2) = e'_1 \wedge e'_2$ for $e_1, e_2 \in \{0, 1\}$. Using Tables 3, 4 and 5, we have

$$\begin{aligned} f(0, 0) &= 0' \wedge 0' = 1 \wedge 1 = 1, & f(0, 1) &= 0' \wedge 1' = 1 \wedge 0 = 0, \\ f(1, 0) &= 1' \wedge 0' = 0 \wedge 1 = 0, & f(1, 1) &= 1' \wedge 1' = 0 \wedge 0 = 0. \end{aligned}$$

We write this information in Table 7.

Table 7: Boolean function for the expression $x'_1 \wedge x'_2$.

e_1	e_2	e'_1	e'_2	$f(e_1, e_2) = e'_1 \wedge e'_2$
0	0	1	1	$1 \wedge 1 = 1$
0	1	1	0	$1 \wedge 0 = 0$
1	0	0	1	$0 \wedge 1 = 0$
1	1	0	0	$0 \wedge 0 = 0$



Boolean Functions(Contd....)

- To get a Boolean expression for a Boolean function h (say), we should first see how many points v_i there are at which $h(v_i) = 0$, and how many points v_i there are at which $h(v_i) = 1$.
- If the number of values for which the function h is 0 is less than the number of values at which h is 1, then we shall choose to obtain the expression in CNF, and not in DNF.
- This will give us a shorter Boolean expression, and hence, a simpler circuit.
- For similar reasons, we will prefer DNF if the number of values at which h is 0 is more.

Boolean Functions(Contd....)



Find the Boolean expressions, in DNF or in CNF (keeping in mind the remark made above), for the functions defined in tabular form below.

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
1	1	1	1
1	1	0	0
1	0	1	0
1	0	0	1
0	1	1	0
0	1	0	0
0	0	1	0
0	0	0	1

x_1	x_2	x_3	$g(x_1, x_2, x_3)$
1	1	1	1
1	1	0	1
1	0	1	0
1	0	0	1
0	1	1	0
0	1	0	0
0	0	1	1
0	0	0	1

Boolean Functions(Contd....)

a) Observe from the given table that, among the two values 0 and 1 of the function $f(x_1, x_2, x_3)$, the value 1 occurs the least number of times. Therefore, by the remark made after **E 13**, we would prefer to obtain the Boolean expression in **DNF**. To get this we will use the stepwise procedure adopted in Example 10.

Accordingly, the required Boolean expression in DNF is given by

$$X(x_1, x_2, x_3) = (x_1 \wedge x_2 \wedge x_3) \wedge (x_1 \wedge x'_2 \wedge x'_3) \vee (x'_1 \wedge x'_2 \wedge x'_3).$$

b) By the given table, among the two values 0 and 1 of the function the points v_i at which $g(v_i) = 0$ are fewer than the points v_i at which $g(v_i) = 1$. So we would prefer to obtain the corresponding Boolean expression in **CNF**.

Applying the stepwise procedure in the solution to E13, the required Boolean expression (in CNF) is given by

$$X(x_1, x_2, x_3) = (x'_1 \vee x_2 \vee x'_3) \wedge (x_1 \vee x'_2 \vee x'_3) \wedge (x_1 \vee x'_2 \vee x_3).$$

Summary

- In this unit, we have considered the following points.
 - The definition and examples of a Boolean expression.
 - The three elementary logic gates, namely, AND-gate, OR-gate and NOT-gate; and the analogy between their functioning and operations of logical connectives.
 - The method of construction of a logic circuit corresponding to a given Boolean expression, and vice-versa.
 - The method of simplifying a Boolean expression

Thank
You



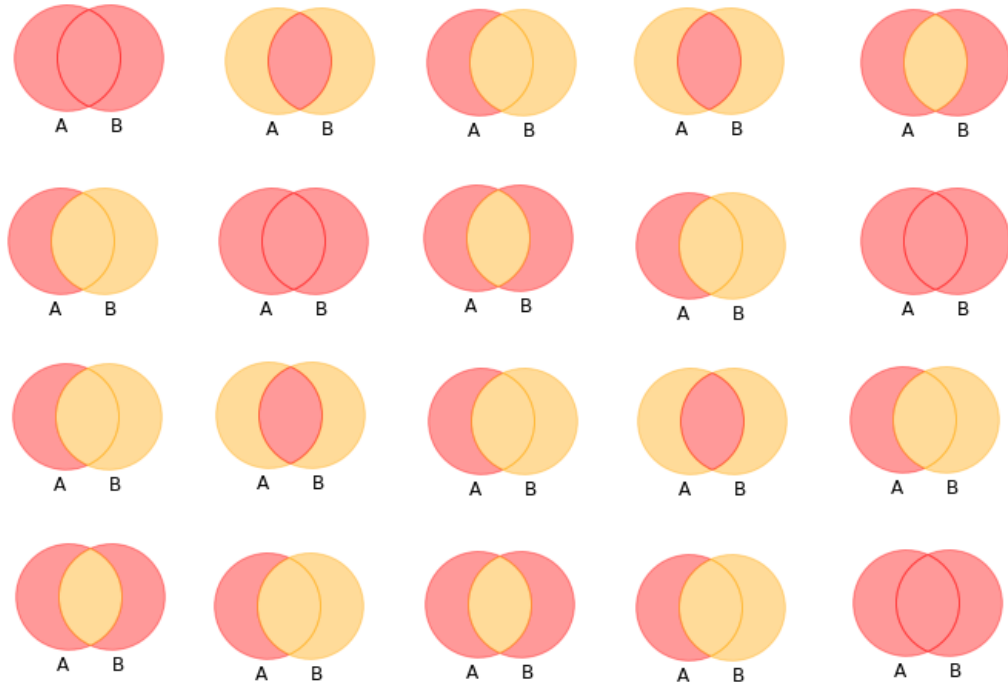
MCS212 Discrete Mathematics

Block 2: Sets and Languages

Unit -1 SETS, RELATIONS AND FUNCTIONS

Topics to be Covered

- Introducing Sets
- Operations on Sets
- Relations
- Function



Introduction

- Various Kind of sets
- 'union' and 'intersection'
- What way Venn diagrams are a useful tool for understanding and working with sets.
- what a relation is, and expose you to some important types of relations.

Introducing Sets

- A set is a well-defined collection. The objects belonging to a set are called elements or members of that set.
- We write this as **$A = \{\text{pen, pencil, eraser, sharpener, paper}\}$**
- There are three ways of representing sets: 'Set-builder form', 'Tabular form' and the pictorial representation through Venn diagrams.

Introducing Sets

- In the '**Set-builder form**', or '**property method**' of representation of sets, we write between brackets $\{ \}$ a variable x , which stands for each of the elements of the set which have the properties $p(x)$, and separate x and $p(x)$ by a symbol ':' or '|' (read as 'such that'). So the set looks like $\{x: p(x)\}$ or $\{x \mid p(x)\}$.
- In '**Tabular form**', or the '**listing method**', the elements of a set are listed one by one within the brackets $\{ \}$, each separated from the other by a comma

Introducing Sets

- A set with no element is called the empty(or null, or void) set, and is denoted by $\emptyset$ or $\{\}$.
 - **For example**, $A = \{x: x \text{ is an integer between 13 and 17 which is divisible by 6}\}$, has no element, i.e., A is the empty set
- A set having a finite number of elements is called a finite set.
 - For example, $\{1,2,4,6\}$ is a finite set because it has four elements, $\emptyset$, the null set, is also a finite set because it has zero number of elements
- A set having infinitely many elements is called an infinite set.
 - For example, the set N of natural numbers is infinite. Similarly, Q , Z , R and C , the set of rational numbers, integers, real numbers and complex numbers, respectively, are infinite set.

Introducing Sets

- A set A is said to be a subset of a set B if each element of A is also an element of B . In this case B is called a superset of A . If A is a subset of B , we represent this by $A \subseteq B$.
- Two sets A and B are equal if every element of A belongs to B and every element of B belongs to A . We represent this by $A=B$.
- A set A is said to be a proper subset of a set B if A is a subset of B and A and B are not equal. We represent this by $A \subset B$.
- The power set of a set A is the set of all the subsets of A , and is denoted by $P(A)$.
- Any set which is a superset of all the sets under consideration is known as the universal set. This is usually denoted by Ω , S or U .

Operations on Sets

- **Basic Operations:**

- **Union:** $A = \{\text{Ram, Mohan, Ravi}\}$ and $B = \{\text{Ravi, Rita, Neetu}\}$, then $A \cup B = \{\text{Ram, Mohan, Ravi, Rita, Neetu}\}$.
- **Intersection:** $A = \{1, 2, 3\}$ and $B = \{2, 1, 5, 6\}$, then $A \cap B = \{1, 2\}$.
- **Difference of two sets:** if $A = \{4, 5, 6, 7, 8, 9\}$ and $B = \{3, 5, 2, 7\}$, then $A \setminus B = \{4, 6, 8, 9\}$
- **Complement of a set:** $U = \{\text{Physics, Chemistry, Mathematics}\}$ and $A = \{\text{Mathematics}\}$, then the complement of A is $A' = \{\text{Physics, Chemistry}\}$.
- **Symmetric difference:** $A \Delta B = (A \setminus B) \cup (B \setminus A)$.

Properties Common to Logic and Sets

i) Associative properties:

Union: $A \cup (B \cap C) = (A \cup B) \cap C$

Intersection: $A \cap (B \cup C) = (A \cap B) \cup C$

ii) Commutative properties:

Union: $A \cup B = B \cup A$

Intersection: $A \cap B = B \cap A$

iii) Identity:

Union: $A \cup \phi = A$

Intersection: $A \cap U = A$

iv) Complement:

Union: $A \cup A' = U$

Intersection: $A \cap A' = \phi$

v) Distributive properties:

Union: $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$

Intersection: $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$

RELATIONS

- Cartesian product: if, $A = \{a, b, c\}$ and $B = \{x, y, z\}$, then
$$A \times B = \{a, b, c\} \times \{x, y, z\} = \{(a, x), (a, y), (a, z), (b, x), (b, y), (b, z), (c, x), (c, y), (c, z)\}$$
- Relations and their Type: if $A = \{1, 2, 3\}$ and $B = \{p, q\}$, then the subset $\{(1, p), (2, q), (2, p)\}$ is a relation on $A \times B$. And $\{(1, 1), (2, 3)\}$ is a relation on A .
- Properties of Relations:
 1. **Reflexive Relations:** if $A = \{1, 2, 3, 4\}$, then the relation $R_1 = \{(1, 1), (2, 2), (3, 3), (4, 4)\}$ in A is reflexive because for $x \in A, (x, x) \in R_1$. However, $R_2 = \{(1, 1), (2, 1), (4, 4)\}$ is not reflexive since $2 \in A$, but $(2, 2) \notin R_2$
 2. **Transitive Relations:** A relation R on a set A is called a transitive relation if whenever $(a, b) \in R$ and $(b, c) \in R$, then $(a, c) \in R$ for $a, b, c \in A$.

RELATIONS

Equivalence Relations: A relation R on a set A is called an **equivalence relation** if and only if

- (i) R is reflexive, i.e., for all $a \in R$, $(a, a) \in R$,
- (ii) R is symmetric, i.e., $(a, b) \in R \Rightarrow (b, a) \in R$, for all $a, b \in A$, and
- (iii) R is transitive, i.e., $(a, b) \in R$ and $(b, c) \in R \Rightarrow (a, c) \in R$, for all $a, b, c \in A$.

FUNCTIONS

- A function from a non-empty set A to a non-empty set B is a subset R of $A \times B$ such that for each $a \in A$ $\exists$ a unique $b \in B$ such that $(a,b) \in R$
- Types of Functions
 - Onto Mapping: A mapping $f: A \rightarrow B$ is said to be onto (or surjective) mapping if $f(A) = B$, that is, the range and co-domain coincide. In this case we say that f maps A onto B
 - Injective Mapping: A mapping $f: A \rightarrow B$ is said to be injective (or one-one) if the images of distinct elements of A under f are distinct, i.e., if $x_1 \neq x_2$ in A , then $f(x_1) \neq f(x_2)$ in B . This is briefly denoted by saying f is 1-1.
 - Bijective Mapping: A mapping $f: A \rightarrow B$ is said to be bijective (or one-one onto,) if f is both injective and surjective, i.e., one-one as well as onto.

*Thank
You*



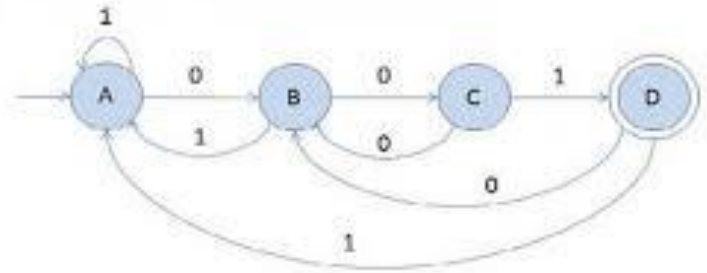
MCS212 Discrete Mathematics

Block 2: Sets and Languages

Unit -2 AUTOMATA AND LANGUAGES

Topics to be Covered

- Regular Expansion
 - Introduction to Defining of Languages
 - Kleene Closure Definition
 - Formal Definition of Regular Expressions
 - Algebra of Regular Expressions
- Regular Language
- Finite Automata
 - Finite Automata
 - Another Method to Describe FA
 - Finite Automata as Output Device
 - Non Deterministic Finite Automata



Regular Expression

- Introduction to Defining of Languages
 - **Alphabet:** A finite set of symbols/characters. We generally denote an alphabet by Σ . If we start an alphabet having only one letter, say, the letter z, then $\Sigma = \{z\}$.
 - **Letter:** Each symbol of an alphabet may also be called a letter of the alphabet or simply a letter.
 - **Language over an alphabet:** A set of words over an alphabet. Languages are denoted by letter L with or without a subscript.
 - **String/word over an alphabet:** Every member of any language is said to be a string or a word.

Regular Expression

- **Example:** Let $L1$ be the language of all possible strings obtained by $L1 = \{z, zz, zzz, zzzz, \dots\}$. This can also be written as $L1 = \{z^n\}$ for $n = 1, 2, 3, \dots$. A string of length zero is said to be null string and is represented by Λ .
- **Kleene Closure Definition:** $\Sigma = \{z\}$, then, $\Sigma^* = L1 = \{\Lambda, z, zz, zzz, \dots\}$
- **Positive Closure:** If we want to modify the concept of closure to refer to only the concatenation leading to non-null strings from a set s , we use the notation $+$ instead of $*$. This plus operation is called positive closure.

Regular Expression

- Formal Definition of Regular Expressions
 - Every letter of Σ can be made into a regular expression $\wedge$ itself is a regular expression.
 - Φ is a regular expression
 - If l and m are regular expressions, then so are
 - (l)
 - lm
 - $l+m$
 - $|^*$
 - $|^+ = ||^*$
 - Nothing else is regular expression.

Regular Expression

- For example, now we would build expression from the symbols 0,1 using the operations of union, concatenation, and Kleene closure.
 - 01 means a zero followed by a one (concatenation)
 - 0+1 means either a zero or a one (union)
 - 0 * means $\Lambda + 0 + 00 + 000 + \dots$ (Kleen closure) .

Expression	Set represented
$(0+1)^*$	all strings over $\{0,1\}$
$0^*10^*10^*$	strings containing exactly two ones
$(0+1)^*11$	strings which end with two ones.

Regular Expression

- Algebra of Regular Expressions

1. $(R+S)+T = R+(S+T)$

2. $R+R = R$

3. $R+\phi = \phi +R = R$

4. $R+S = S+R$

5. $R\phi = \phi R = \phi$

6. $R\Lambda = \Lambda R = R$

7. $(RS)T = R(ST)$

8. $R(S+T) = RS+RT$

9. $(S+T)R = SR+TR$

10. $\phi^* = \Lambda^* = \Lambda$

11. $R^*R^* = R^* = (R^*)^*$

12. $RR^* = R^*R = R^* = \Lambda + RR^*$

13. $(R+S)^* = (R^*S^*)^* = (R^*+S^*)^* = R^*S^* = (R^*S)^*R^* = R^*(SR^*)^*$

14. $(RS)^* = (R^*S^*)^* = (R^*+S^*)^*$

Regular Language

- Regular language is a language that can be represented by a regular expression.
 - **Rule 1:** $\phi, \{\Lambda\}$ and $\{a\}$ are regular languages denoted respectively by regular expressions ϕ and Λ .
 - **Rule 2:** For each a in Σ , the set $\{a\}$ is a regular language denoted by the regular expression a .
 - **Rule 3:** If l is a regular expression associated with the language L and m is a regular expression associated with the language M , then
 - lm
 - $(l+m) = L \cup M$
 - $(l^*) = L^*$

Finite Automata

- A finite automata is similar to a finite state machine. A finite automata consists of five parts:
 - a finite set of states;
 - a finite set of alphabets;
 - an initial state;
 - a subset of set of states (whose elements are called “yes” state or; accepting state;) and

- a next-state function or a transition state function.

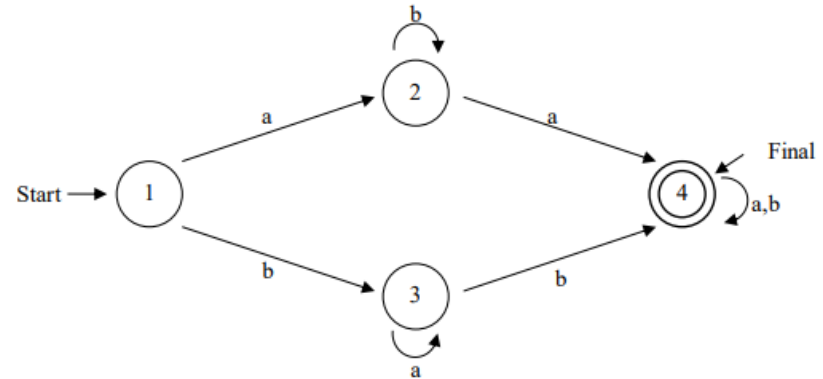
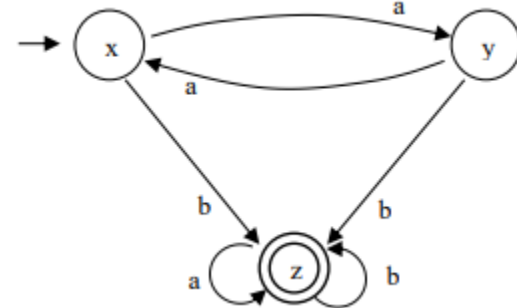


Fig. Finite Automata

Finite Automata

State	Input	
	a	b
Start x	y	z
y	x	z
Final z	z	z



Another Method to Describe FA

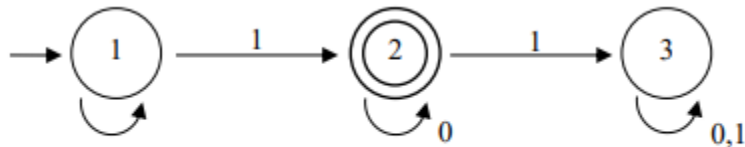


Fig. 1 Finite automata

	State	Input		Accept?
		0	1	
Start	1	1	2	No
Final	2	2	3	Yes
	3	3	3	No

Finite Automata

- **Mealy Machine** – A mealy machine is defined as a machine in theory of computation whose output values are determined by both its current state and current inputs. In this machine atmost one transition is possible.

It has 6 tuples: $(Q, q_0, \Sigma, O, \delta, \lambda')$

Q is finite set of states

q_0 is the initial state

Σ is the input alphabet

O is the output alphabet

δ is transition function which maps $Q \times \Sigma \rightarrow Q$

' λ' is the output function which maps $Q \times \Sigma \rightarrow O$

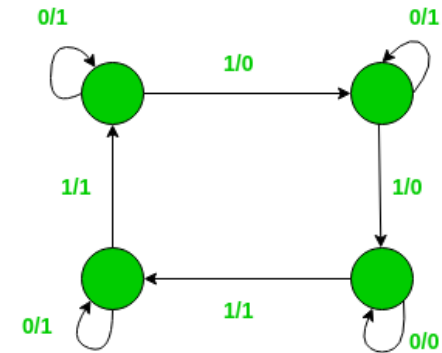


Figure - Mealy machine

Finite Automata

- A moore machine is defined as a machine in theory of computation whose output values are determined only by its current state. It has also 6 tuples: $(Q, q_0, \Sigma, O, \delta, \lambda)$
 - Q is finite set of states
 - q_0 is the initial state
 - Σ is the input alphabet
 - O is the output alphabet
 - δ is transition function which maps $Q \times \Sigma \rightarrow Q$
 - λ is the output function which maps $Q \rightarrow O$

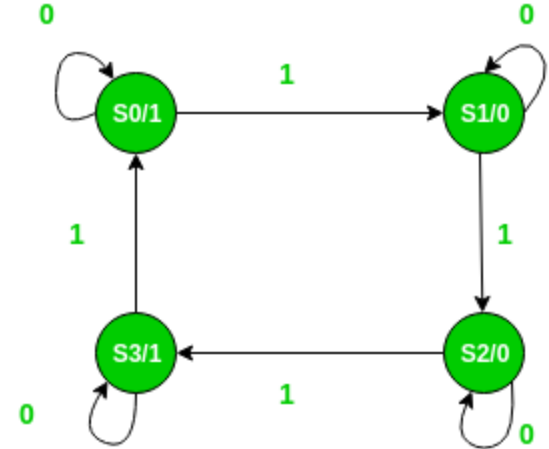
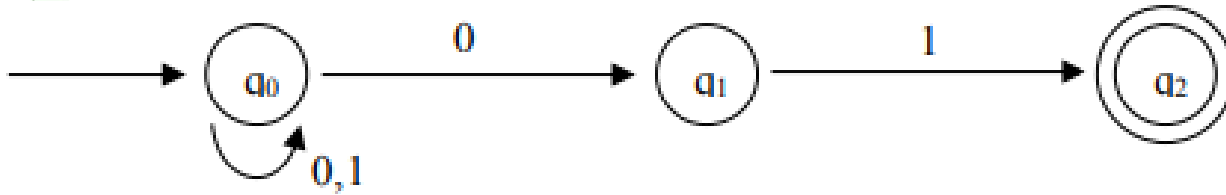
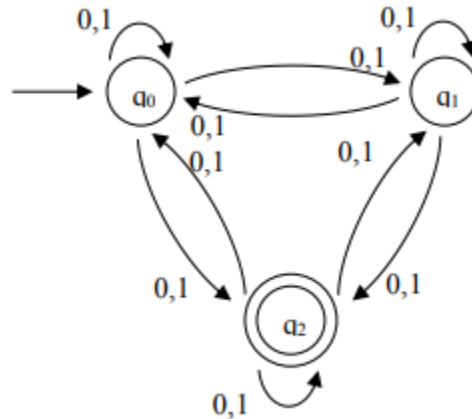


Figure - Moore machine

Non Deterministic Finite Automata



- (i) On input 0 in state q_0 , the next state may be either of the two states viz., q_0 or q_1 . (ii) There is no next state on input 0 in the state q_1 . (iii) There is no next state on input 0 and 1 in the state q_2 .



*Thank
You*



MCS212 Discrete Mathematics

Block 2: Sets and Languages

Unit -3 Computability And Complexity

Topics to be Covered

- Turing Machine
- Nondeterministic Turing Machine
- Undecidable and Halting Problems
- Complexity



Turing Machine

- A Turing Machine (TM) is a mathematical model which consists of an infinite length tape divided into cells on which input is given.
- It consists of a head which reads the input tape.
- A state register stores the state of the Turing machine.
- After reading an input symbol, it is replaced with another symbol, its internal state is changed, and it moves from one cell to the right or left.
- If the TM reaches the final state, the input string is accepted, otherwise rejected.

Turing Machine

- A TM can be formally described as a 7-tuple $(Q, X, \Sigma, \delta, q_0, B, F)$ where
- **Q** is a finite set of states
- **X** is the tape alphabet
- Σ is the input alphabet
- δ is a transition function; $\delta : Q \times X \rightarrow Q \times X \times \{\text{Left_shift}, \text{Right_shift}\}$.
- **q₀** is the initial state
- **B** is the blank symbol
- **F** is the set of final states

Turing Machine

- Transition Diagrams:** A Transition Diagram of the next-move functions δ of a TM is a graphical representation consisting of a finite number of nodes and (directed) labelled arcs between the nodes.

Let $M = \{Q, \Sigma, \Gamma, \delta, q_0, h\}$

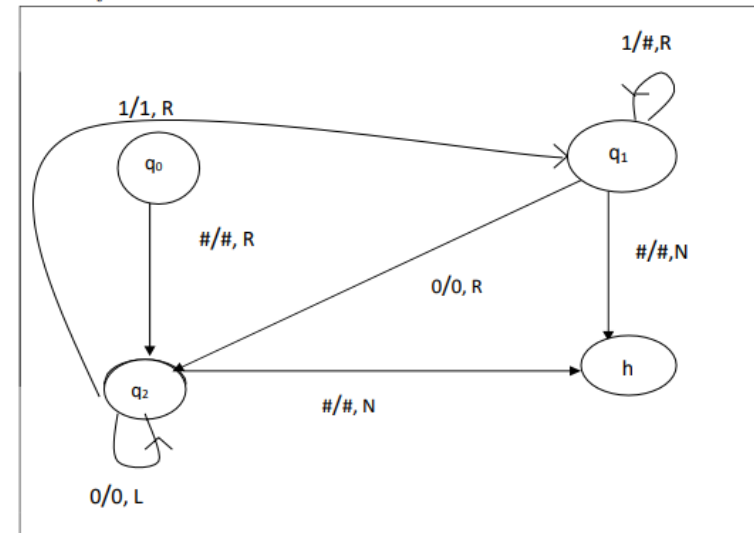
Where $Q = \{q_0, q_1, q_2, h\}$

$\Sigma = \{0, 1\}$

$\Gamma = \{0, 1, \#\}$

and δ be given by the following table.

	0	1	#
q_0	-	-	$(q_2, \#, R)$
q_1	$(q_2, 0, R)$	$(q_1, \#, R)$	$(h, \#, N)$
q_2	$(q_2, 0, L)$	$(q_1, 1, R)$	$(h, \#, N)$
H	-	-	-



Some Formal definitions

- **Turing Acceptable Language:** A language L over some alphabet is said to be Turing Acceptable Language, if there exists a Turing Machine M such that $L = L(M)$.
- **Turing Decidable Language:** A language over some alphabet is said to be Turing acceptable language if there exists a Turing Machine M such that $L = L(M)$ and M halts on every input.
- For a TM M and an input string $w \in \Sigma^*$, even after a large number of moves we may not reach the halt state. However, from this we can neither conclude that 'Halt state will be reached in a finite number of moves' nor can we conclude that Halt state will not be reached in a finite number moves.
- This raises the question of how to decide that an input string w is not accepted by a TM M .

Nondeterministic Turing Machine

- In a Non-Deterministic Turing Machine, for every state and symbol, there are a group of actions the TM can have. So, here the transitions are not deterministic. The computation of a non-deterministic Turing Machine is a tree of configurations that can be reached from the start configuration.
- An input is accepted if there is at least one node of the tree which is an accept configuration, otherwise it is not accepted. If all branches of the computational tree halt on all inputs, the non-deterministic Turing Machine is called a **Decider** and if for some input, all branches are rejected, the input is also rejected.

Nondeterministic Turing Machine

- A non-deterministic Turing machine can be formally defined as a 6-tuple $(Q, X, \Sigma, \delta, q_0, B, F)$ where –
- **Q** is a finite set of states
- **X** is the tape alphabet
- Σ is the input alphabet
- δ is a transition function;
- $\delta : Q \times X \rightarrow P(Q \times X \times \{\text{Left_shift}, \text{Right_shift}\})$.
- **q₀** is the initial state
- **B** is the blank symbol
- **F** is the set of final states

Undecidable and Halting Problems

- **UNDECIDABLE PROBLEM:** A function is said to be un-computable if no such machine exists. There may be a Turing machine that can compute f on part of its domain, but we call the function computable only if there is a Turing machine that computes the function on the whole of its domain.
- **THE HALTING PROBLEM:** There are many problems which are not computable. But, we start with a problem which is important and that at the same time gives us a platform for developing later results. One such problem is the halting problem. Algorithms may contain loops that may be infinite or finite in length. The amount of work done in an algorithm usually depends on the data input. Algorithms may consist of various numbers of loops, nested or in sequence.

Complexity

- The complexity of an algorithm computes the amount of time and spaces required by an algorithm for an input of size (n).
- The complexity of an algorithm can be divided into two types. The **time complexity** and the **space complexity**.
- **What is Space complexity?**
- When we design an algorithm to solve a problem, it needs some computer memory to complete its execution. For any algorithm, memory is required for the following purposes...

Complexity

- To store program instructions.
- To store constant values.
- To store variable values.
- And for few other things like function calls, jumping statements etc,.

- **What is Time complexity?**

- Every algorithm requires some amount of computer time to execute its instruction to perform the task. This computer time required is called time complexity.

Complexity

<code>int sumOfList(int A[], int n)</code>	Cost Time require for line (Units)	Repeataation No. of Times Executed	Total Total Time required in worst case
{			
<code>int sum = 0, i;</code>	1	1	1
<code>for(i = 0; i < n; i++)</code>	1 + 1 + 1	1 + (n+1) + n	2n + 2
<code>sum = sum + A[i];</code>	2	n	2n
<code>return sum;</code>	1	1	1
}			
4n + 4 Total Time required			

In above calculation

Cost is the amount of computer time required for a single operation in each line.

Repeataation is the amount of computer time required by each operation for all its repeataations.

Total is the amount of computer time required by each operation to execute. So above code requires '**4n+4**' **Units** of computer time to complete the task. Here the exact time is not fixed. And it changes based on the **n** value. If we increase the **n** value then the time required also increases linearly.

*Thank
You*



V-LRN

VIRTUAL LEARNING NETWORK

Discrete Mathematics

Block – 3

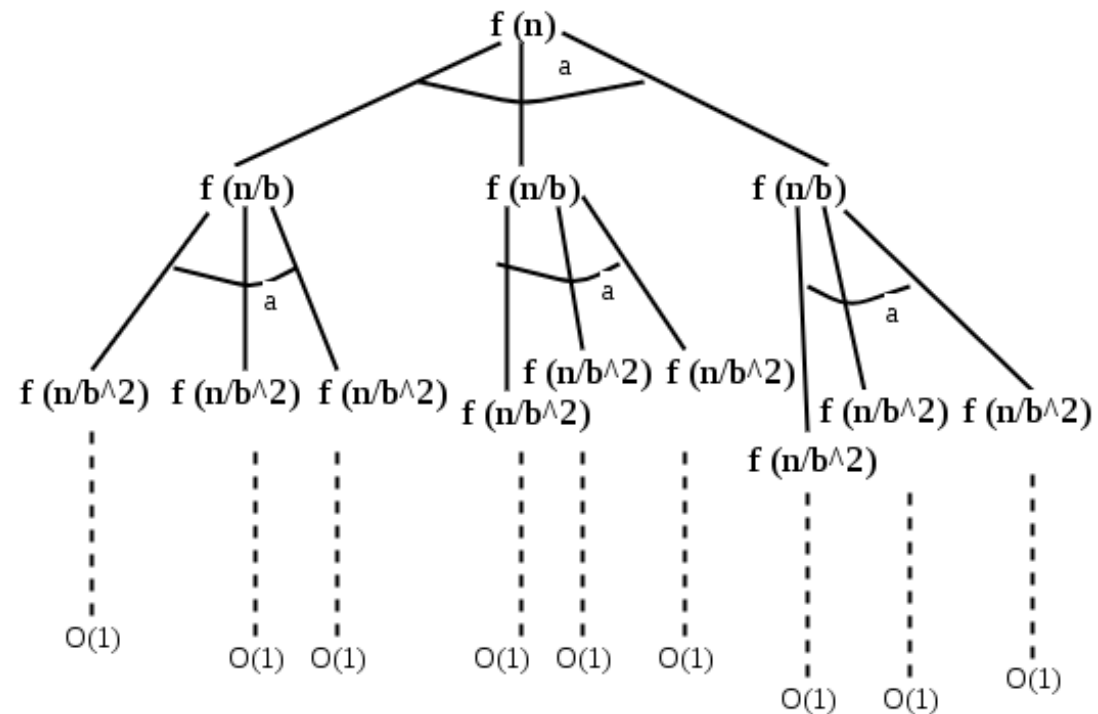
Unit - 3

Recurrence Relations

Virtual Learning Network

Topics to be Covered

- Introduction
- Three Recurrent Problems
- More Recurrences
- 4 Divide and Conquer Technique to Solve Recurrences
- Summary





Introduction

- Denote the number of such binary strings by a_n , then $a_1 = 2$ and $a_2 = 3$.
- Also, we can show that $a_n = a_{n-1} + a_{n-2}$ for $n \geq 2$. This is an example of a recurrence relation.
- This relates the value of a_n , a_{n-1} and a_{n-2} . We will show how to find an explicit expression for a_n using this relation in units 2 and 3.
- In this unit, we will discuss how to formulate such recurrence relations for solving combinatorial problems.

Three Recurrent Problems

- The procedure for finding the terms of a sequence in a recursive manner is called **Recurrence Relation**.
- A recurrence relation is an equation that recursively defines a sequence where the next term is a function of the previous terms (Expressing F_n as some combination of F_i with $i < n$).
- **Example-** Fibonacci series – $F_n = F_{n-1} + F_{n-2} = F_{n-1} + F_{n-2}$,
- Tower of Hanoi – $F_n = 2F_{n-1} + 1$

Three Recurrent Problems(Contd....)

Table 1 : Finding the slack permutation corresponding to the Admissible sequence $(+, -, +, +, -, -, +, +, -)$

Sequence	Input String	Stack	Output String
+ (Push 4)	123	[4]	Empty
- (Pop 4)	123	[]	4
+ (Push 3)	12	[3]	4
+ (Push 2)	1	[23]	4
- (Pop 2)	1	[3]	24
- (Pop 3)	1	[]	324
+ (Push 1)	Empty	[1]	324
- (Pop 1)	Empty	[]	1324

More Recurrences

Example 7: You may recall that the set of all subsets of any non-empty set, S , is called its **power set**, and denoted by $p(S)$. Let us determine a recurrence relation satisfied by $s_n = |p(S)|$, where $|S| = n$. Let us take $S = \{1, 2, \dots, n\}$. Now, any subset, A , of S either contains the number n or does not contain the number n . Let us consider these two mutually exclusive cases separately and count the number of such subsets, A . If $n \in A$, then $A = A' \cup \{n\}$, where A' is a subset of $\{1, 2, \dots, n-1\}$, there are s_{n-1} such subsets A . So, the number of subsets A that contain n is the same as the number of subsets of $\{1, 2, \dots, n-1\}$, which is s_{n-1} . On the other hand if $n \notin A$, then, in fact, A is a subset of $\{1, 2, \dots, n-1\}$, and there are s_{n-1} of these too. Combining these, we see that $s_n = s_{n-1} + s_{n-1} = 2s_{n-1}$, $n \geq 1$, with $s_0 = 1$.



Divide and Conquer Technique to Solve Recurrences

- Often, to solve a problem, we can partition the problem into smaller problems, find solutions to the smaller problems and then combine the solutions to find a solution for the whole.
- We can repeatedly partition the problem till we reach a stage where we can solve the problem quite easily.
- This approach is called the divide-and-conquer approach.
- Let us start with an example. Throughout this section we will assume that $n = 2^k$.



Divide and Conquer Technique to Solve Recurrences (Contd....)

Example 10: Consider the problem of finding the maximum and minimum of a list of n numbers. Here let us assume that $n = 2^k$ for some k to make things simpler for us.

Let a_n be the number of comparisons required for finding the maximum and minimum of a list of n numbers. We can partition the list into two lists of size $\frac{n}{2}$ each. The maximum and minimum of each of the list can be found using $a_{\frac{n}{2}}$ comparisons. We can then compare minimum (resp. maximum) of the lists to get the minimum (resp. maximum) of the list, i.e., we will need two more comparisons. So, $a_n = 2 a_{\frac{n}{2}} + 2$ for $n \geq 2$. $a_2 = 2$. We leave it as an exercise for you to check that $a_n = \frac{3}{2}n - 2$, when n is a power of 2.

Summary

- In this Unit recurrence relations
 - several examples of recurrence relations, drawn from well-known problems and from routine exercises in combinatorics.
 - how to set up recurrence relations after having read this unit.
 - setting up of recurrence relations with the help of divide and conquer paradigm.

Thank
You



V-LRN

VIRTUAL LEARNING NETWORK

Discrete Mathematics

Block – 3

Unit - 4

Partitions and Distributions

Virtual Learning Network

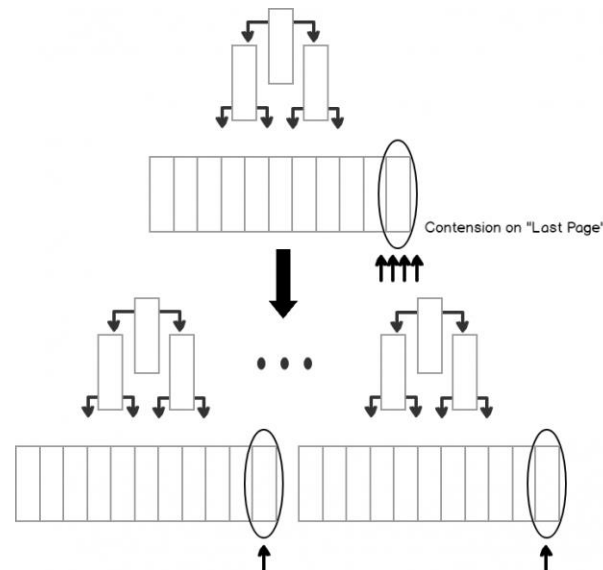


Topics to be Covered

- Introduction
- Integer Partitions
- Distributions
 - Distinguishable Objects into Distinguishable Containers
 - Distinguishable Objects into Indistinguishable Containers
 - Indistinguishable Objects into Distinguishable Containers
 - Indistinguishable Objects into Indistinguishable Containers
- Summary

Introduction

- There are four broad possible kinds of distributions. In each case, we consider ways of counting all the distributions.
- In the process you will also be introduced to Stirling numbers.
- With this unit we come to the end of our discussion on counting techniques.



Integer Partitions

Definition: Any representation of $n \in \mathbf{N}$ as a sum of positive integers in non-increasing order is called a **partition** (or **integer partition**) of n . Each such partition can be written in the form $n = p_1 + p_2 + \dots + p_k$, where $p_1 \leq p_2 \leq \dots \leq p_k$.

Here, $p_1, p_2, \dots, p_k$ are called the **parts** of the partition, and the **number of parts** of the partition is k .

- While we chose 100 in the example above, it is really a huge number in the context of integer partitions. Let us consider a smaller number, say 5. How many partitions of 5 can you think of? There are 7 altogether, namely, 5, 1+4, 2+3, 1+1+3, 1+2+2, 1+1+1+2 and 1+1+1+1+1.

If we represent the number of partitions of the integer n by P_n , we have shown that $P_5 = 7$. These partitions have 1, 2, 2, 3, 3, 4 and 5 parts, respectively.

If we represent the number of partitions of n with exactly k parts by P_n^k , then we have $P_5^1 = 1, P_5^2 = 2, P_5^3 = 2, P_5^4 = 1, P_5^5 = 1$.



Distributions

- Distinguishable Objects into Distinguishable Containers.
 - **Example:** Show that the number of words of length n on an alphabet of m letters is m^n .
 - **Solution:** The m letters of the alphabet can be used any number of times in a word of n letters. The word can be considered as n ordered boxes, each holding a letter from the alphabet. The boxes become distinguishable because they are 'ordered'. The letters of the alphabet are clearly distinguishable. So, the number of ways of doing this is m^n .

Distributions(Contd....)

- Distinguishable Objects into Indistinguishable Containers.

Definition: For natural numbers n and m , the **Stirling number of the second kind**, S_n^m , is the number of partitions of an n -element set into exactly m parts.

Note that:

- $S_n^m = 0$ if $n < m$, for, if the number of containers exceeds the number of objects, then it is impossible to have all the containers non-empty.
- $S_n^n = 1$, since there is only one way of putting n distinguishable objects in n indistinguishable boxes so that no box is empty.
- $S_n^1 = 1$.



Distributions(Contd....)

- **Indistinguishable Objects into Distinguishable Containers.**
- Example 5: How many distinct solutions are there of $x + y + z + w = 10$
 - i) in non-negative integers?
 - ii) in positive integers?
- From the result quoted above, the answer is $C(4 + 10 - 1, 10) = 286$.
- ii) We want x, y, z, w to be positive. Hence, we can write them respectively as $X+1, Y+1, Z+1, W+1$, where X, Y, Z, W are non-negative.

Distributions(Contd....)

- Hence we want the number of non-negative solutions of the equation $X+1+Y+1+Z+1+W+1=10$, i.e., $X+Y+Z+W=6$. The answer, now, is $C(4 + 6 - 1, 6) = 84$.
- **Indistinguishable Objects into Indistinguishable Containers**
- **Example:** In how many ways can 20 identical books be placed in 4 identical boxes?

Solution: The answer is $P_{20}^1 + P_{20}^2 + P_{20}^3 + P_{20}^4 = P_{24}^4$

Summary

i) n^m , if the objects and containers are distinguishable.

ii) $\sum_{i=1}^m S_n^i$, if the objects are distinguishable but the containers are not.

(Here S_j^i is a Stirling number of the second kind).

iii) $C(m+n-1, n)$ if the objects are not distinguishable but the containers are distinguishable.

iv) P_{n+m}^m , if neither the objects nor the containers are distinguishable.

Thank
you



V-LRN

VIRTUAL LEARNING NETWORK

Discrete Mathematics

Block – 4

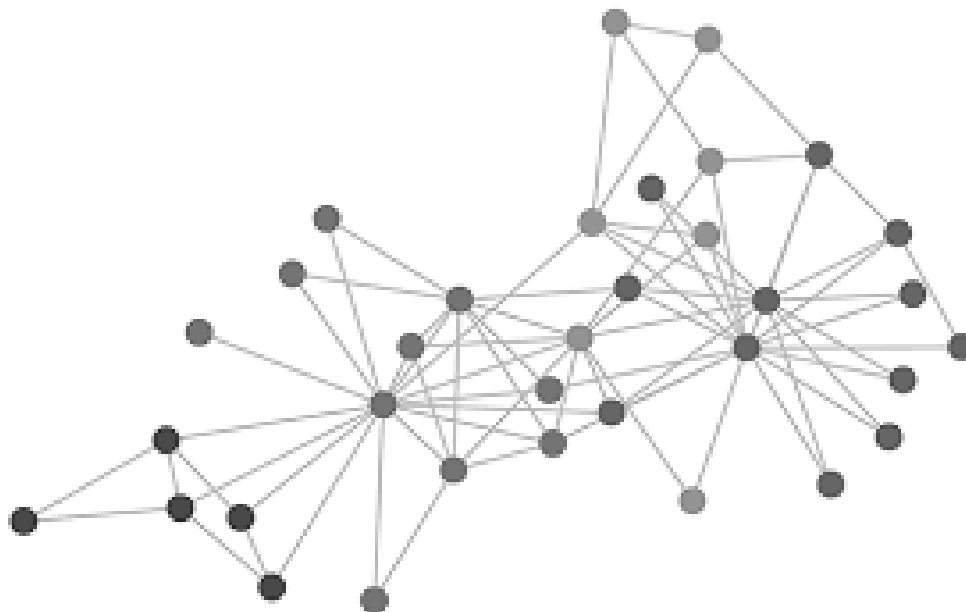
Unit - 1

Basic Properties of Graphs

Virtual Learning Network

Topics to be Covered

- Introduction
- Graphs
- Degree, Regularity and Isomorphism
- Subgraphs



Introduction

- Example - Public transport system in a city. Various places are the objects here, the bus routes are the connections, and we need to know the places connected to the railway station, say.
- Yet another problem could be that of establishing communication links between different centres.
- The solutions to the given problems can be obtained by analysing their graphs.
- Ideas given by various mathematicians to solve such problems gave birth to a branch of mathematics called **graph theory**.

Graphs

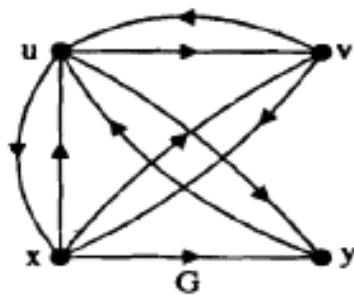
- A graph is a set of points, called nodes or vertices, which are interconnected by a set of lines called edges.
- A graph (denoted as $G=(V,E)$) consists of a non-empty set of vertices or nodes V and a set of edges E .
- An **undirected graph** G is a finite non-empty set V together with a set E consisting of pairs of points of V .
- The set V is called the vertex set of G , the set E is called the edge set of G .
- To show the relationship between V , E and G , we write $G = (V(G), E(G))$.

Graphs(Contd....)

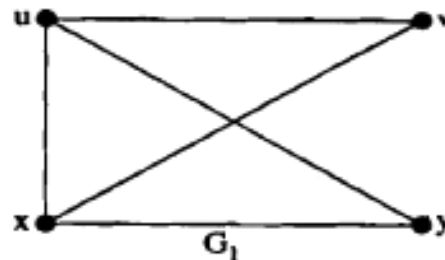
- A directed graph, or digraph, G consists of a finite non-empty set V together with a subset E of the Cartesian product set $V \times V$.
- We call V the vertex set of G and E the edge set of G , and we write $G = (V(G), E(G))$.
- **Note** that, in an undirected graph, if (u, v) is an edge, so is (v, u) . So, the relation $E(G)$ is a symmetric binary relation on $V(G)$.
- Sometimes, it may happen that in a graph there is a loop, i.e., an edge joins a vertex to itself as in Fig.4 (c)[**Given in next slide**] where (u, u) shows a loop.

Graphs(Contd....)

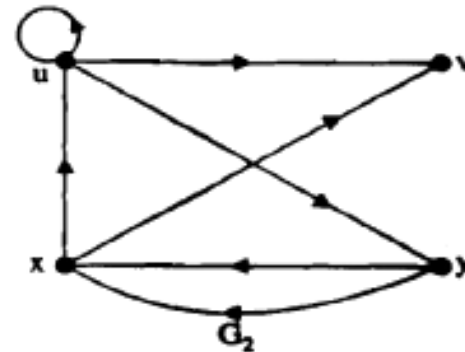
- It may also happen that there are two or more edges joining the same vertices, as in Fig.4 (c), where there are two edges joining y to x .
- Such edges are called parallel or multiple edges, and a graph with any parallel edges is called a multigraph.



(a)



(b)



(c)

- An undirected graph without loops or parallel edges is called a simple graph.

Graphs(Contd....)

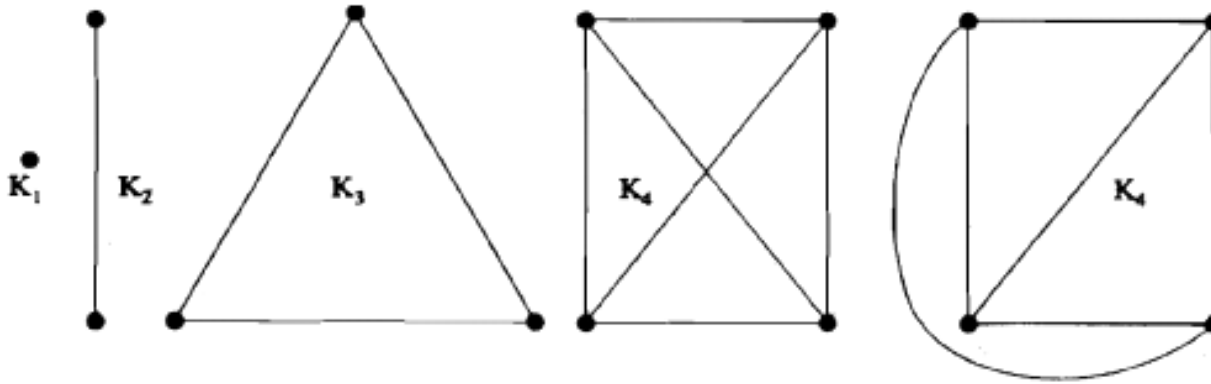
- **Terminology**

- If e is an edge joining the vertices u and v of a graph $G=(V,E)$, we will denote it as uv . In this case, u and v are called adjacent vertices (or neighbours), and are the endpoints of e . We also say that e is incident with u and v .
- If distinct edges e_1 and e_2 of G have at least one vertex in common, then e_1 and e_2 are called adjacent edges.

Graphs(Contd....)

- For an example of these concepts, consider the graph G_1 . $G_1 = (V, E)$, where $V = \{u, v, x, y\}$ and $E = \{uv, ux, uy, vx, xy\}$.
- So, G_1 is a (4, 5)-graph. The only nonadjacent vertices of G_1 are v and y .
- The edges uv and vx are adjacent, since both are incident with the vertex v .
- A **complete graph** is a graph in which any two vertices are adjacent, i.e., each vertex is joined to every other vertex by an edge.
- We denote the complete graph on n vertices by K_n

Graphs(Contd....)

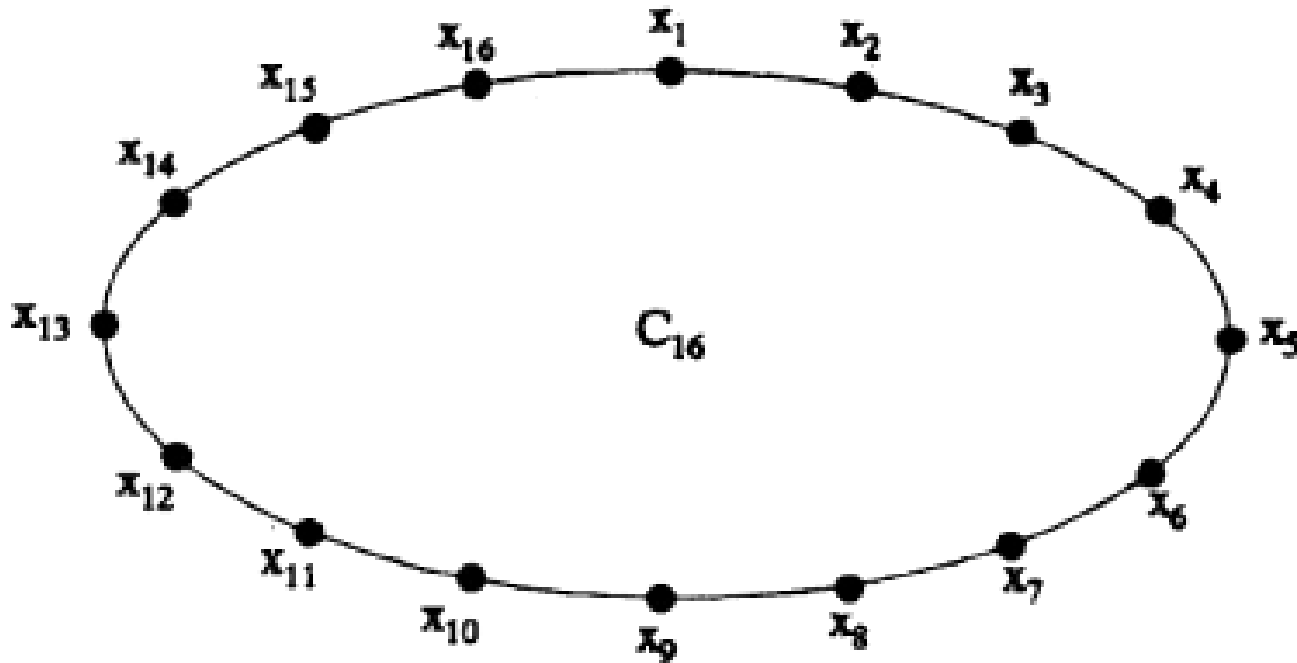


- **Star topology:** Various computers, printers and plotters on a campus can be connected using a local area network. Some of these networks are based on graphs like the one given in Fig, called a star topology



Graphs(Contd....)

- **Cycles:** A cycle C_n is a graph on n vertices $\{x_1, \dots, x_n\}$ where $E(C_n) = \{x_i x_{i+1} : 1 \leq i \leq n-1\} \cup \{x_n x_1\}$. For instance, C_{16} is shown in Fig

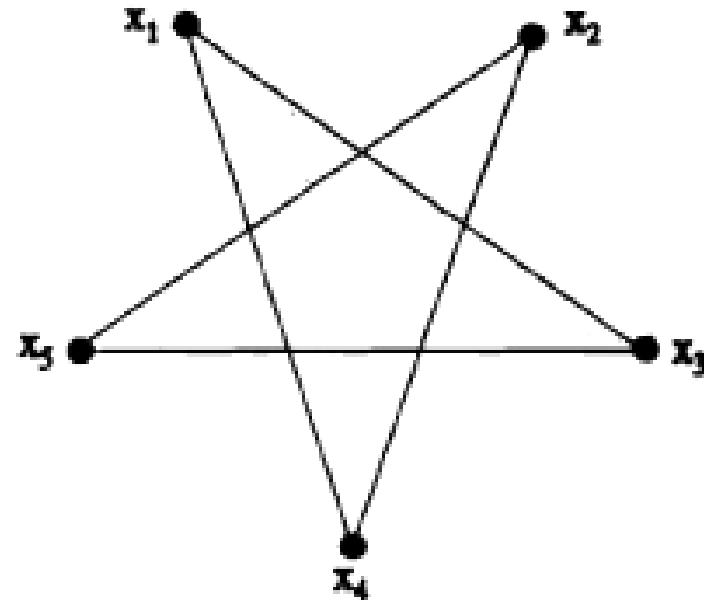
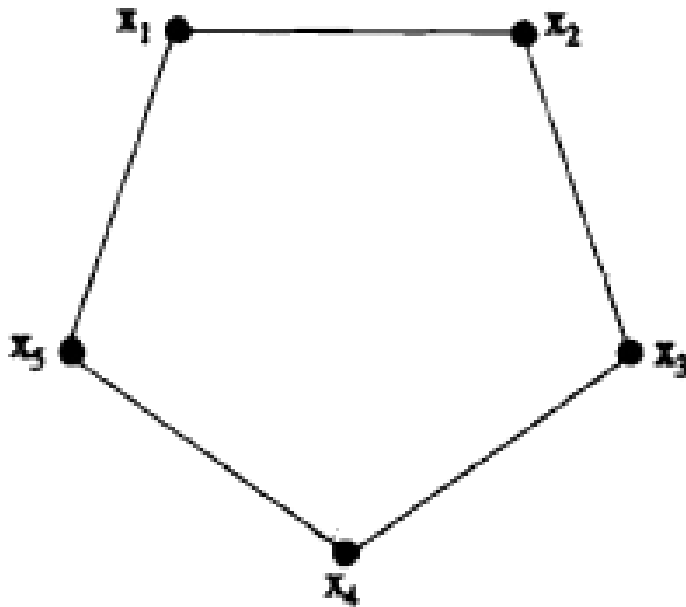


Graphs(Contd....)

- Let $G = (V, E)$ be a (p, q) -graph.
- By its complement $\overline{G}$, we mean the graph with $V(\overline{G}) = V(G)$ and $E(\overline{G}) = \{xy : xy \notin E(G), x, y \text{ belongs to } V(G)\}$.
- Note that $\overline{G}$ is a $(p, \overline{q})$ -graph, where $\overline{q} = (\text{number of pairs of elements of } V) - q$.
- Since in a set V with p elements, there can be $C(p, 2) = p(p-1)/2$ such pairs of elements, $\overline{q} = 2 \cdot p(p-1)/2 - q$.

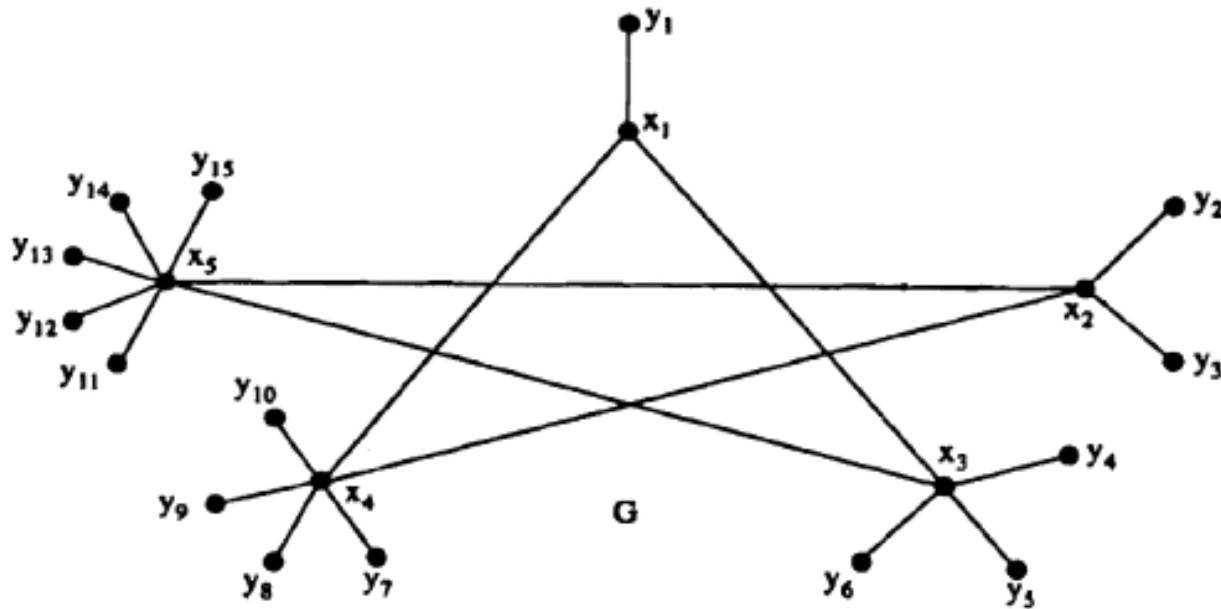
Graphs(Contd....)

- C_5 and its complement. Check whether C_5 is also a cycle or not.



Degree, Regularity And Isomorphism

- The degree of a vertex x in G , we mean the number of edges incident with x . We denote the degree of x by $d_G(x)$.
- Example : Consider the graph G shown in Fig. First consider the vertex x_1 . Clearly, three edges are incident with it, so that $d(x_1) = 3$. Likewise you may observe that $d(x_2) = 4$, $d(x_3) = 5$, $d(x_4) = 6$ and $d(x_5) = 7$.



Degree, Regularity And Isomorphism

Theorem 1 (Handshaking Theorem) : If G is a (p, q) -graph with

$V(G) = \{v_1, \dots, v_p\}$, and if $d_i = d_G(v_i)$, $1 \leq i \leq p$, then $2q = \sum_{i=1}^p d_i$.

That is, the sum of the degrees of the vertices of G is twice the number of edges.

Proof: Consider the set $S = \{(x, e) : x \in V(G), e \in E(G), x \text{ is an endpoint of } e\}$. Choose a vertex $v_i \in V$. This can be done in p ways. Now, since $d_i = d(v_i)$, there are precisely d_i edges incident with this vertex v_i . These edges give d_i elements of the set S . Adding over all the vertices of G , we get

$$|S| = \sum_{i=1}^p d_i. \quad (1)$$

Degree, Regularity And Isomorphism

Now choose an edge e in $E(G)$. This can be done in q ways. This edge has precisely two endpoints, and they give two elements of S . Summing over every edge $e \in E(G)$, we get

$$|S| = 2q \quad (2)$$

This is because every edge is counted twice, once for each vertex it contains. Equating (1) and (2) we get the required result.

Corollary 1 : The sum of the degrees of all the vertices of any graph is even.

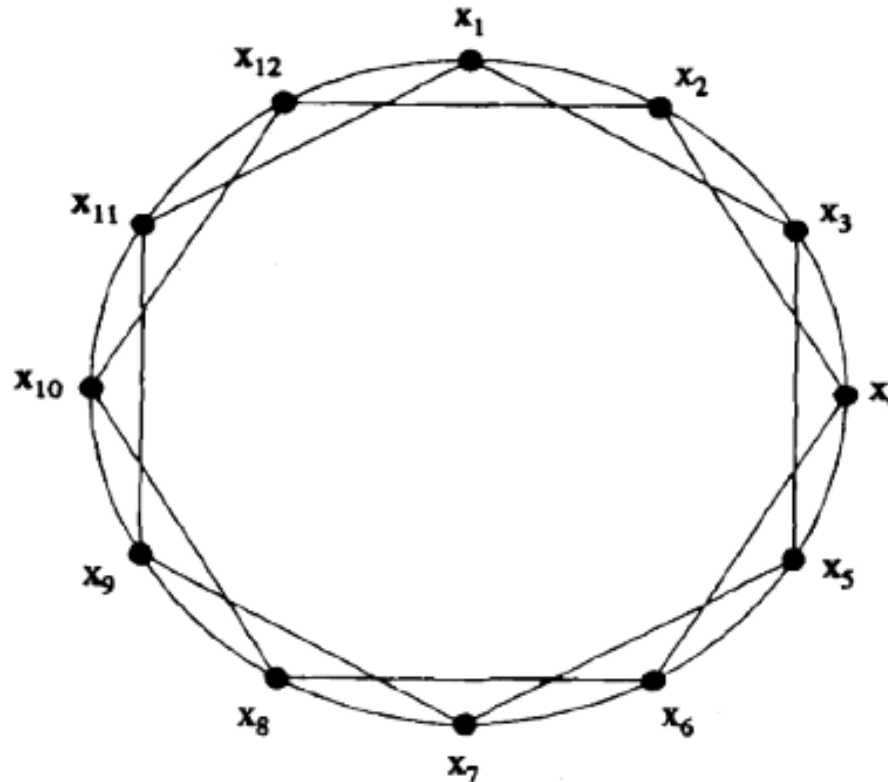
Corollary 2 : Any graph can only have an even number of odd vertices.

Corollary 3 : At any party, the number of people who shake the hands of an odd number of people is even.

Degree, Regularity And Isomorphism

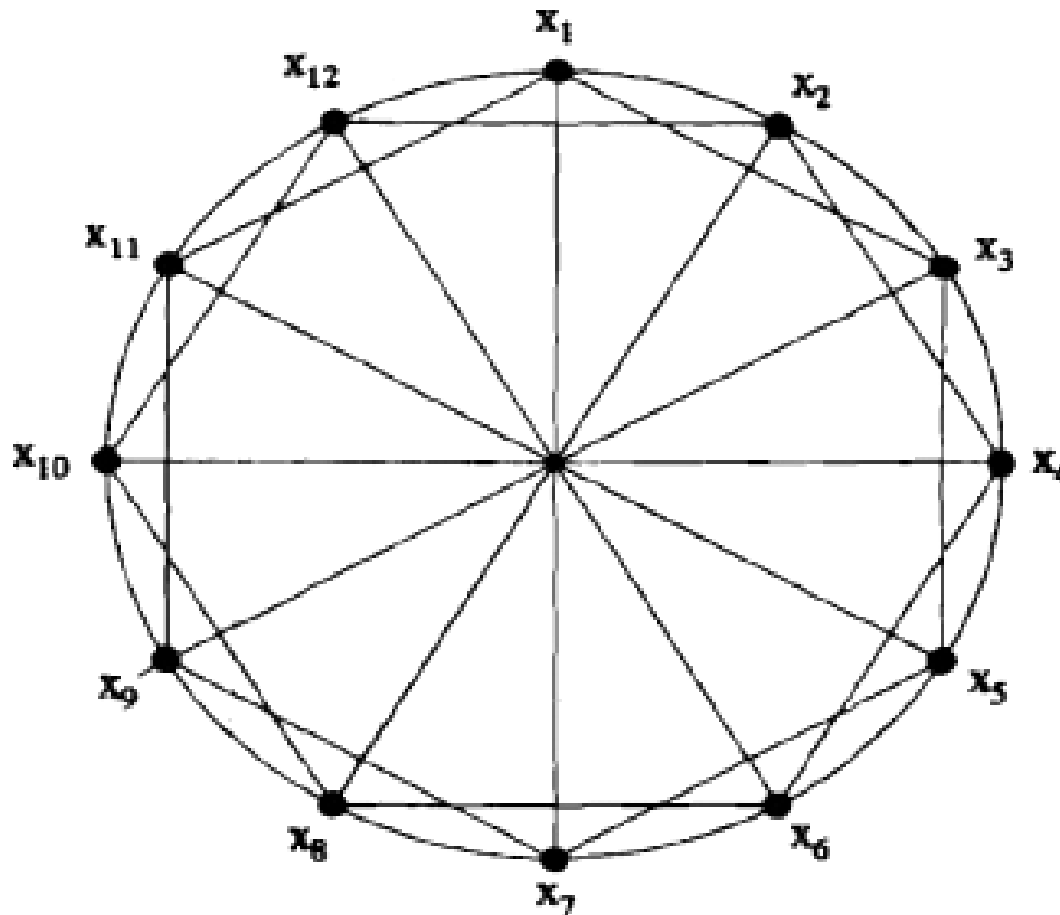
Definition: A (p, q) -graph G is said to be **regular**, with **degree of regularity** r , if $d_G(x) = r$ for every vertex $x \in V(G)$. In this case we also say that G is an **r -regular graph**. Of course, $0 \leq r \leq (p-1)$.

- Construct a 4-regular graph G with 12 vertices



Degree, Regularity And Isomorphism

- Construct a 5-regular graph on 12 vertices now.

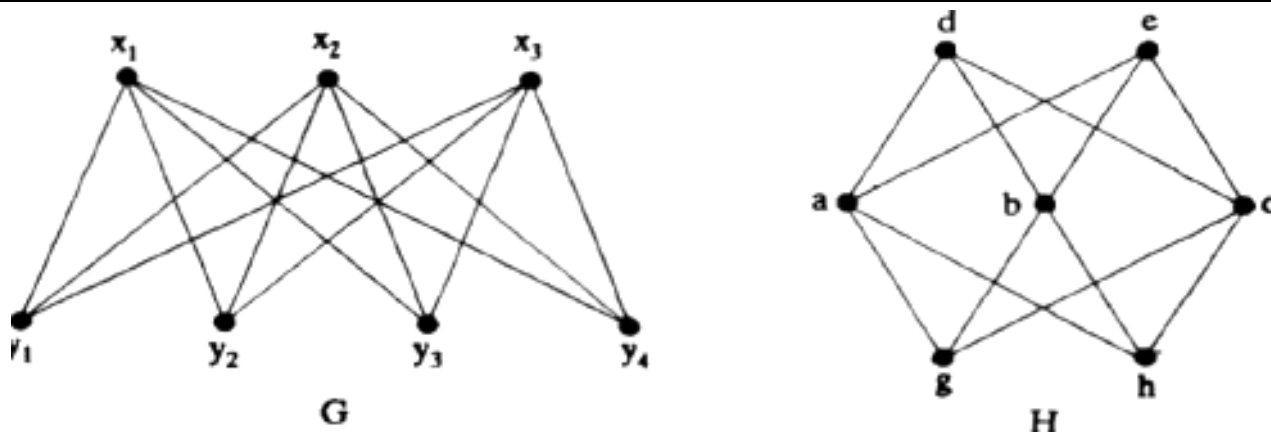




Degree, Regularity And Isomorphism

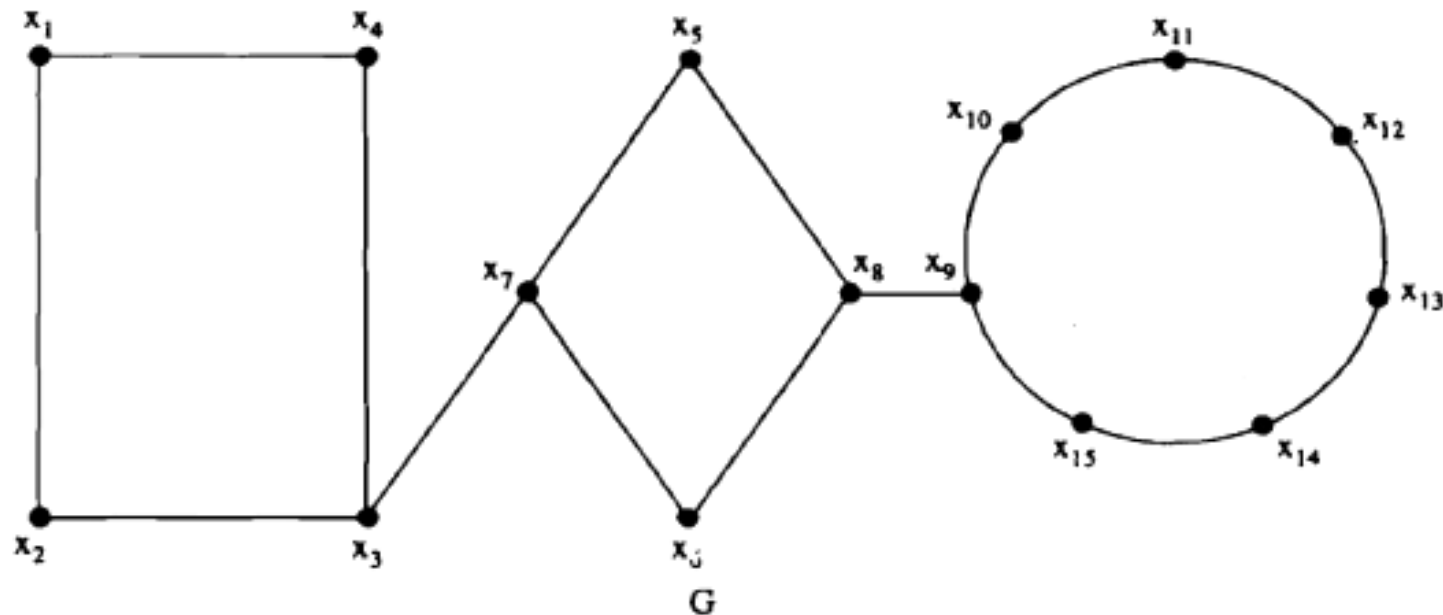
- Let $G = (V(G), E(G))$, $H = (V(H), E(H))$ be two graphs.
- By an isomorphism f from the graph G to the graph H , we mean a map $f: V(G) \rightarrow V(H)$ such that
 - f is one-one and onto; and
 - $x y$ belongs to $E(G)$ if and only if $f(x) f(y)$ belongs to $E(H)$.
- In this case we say that G and H are isomorphic. Otherwise they are called non isomorphic.

Degree, Regularity And Isomorphism



- Define a map $f : V(G) \rightarrow V(H)$ as follows:
- $f(x_1) = a, f(x_2) = b, f(x_3) = c, f(y_1) = d, f(y_2) = e, f(y_3) = g, f(y_4) = h$.
- Observe that uv belongs to $E(G)$ if and only if $f(u)f(v)$ belongs to $E(H)$.
- The two graphs shown in Fig. are isomorphic under this correspondence.

Subgraphs



Let $V(G_1) = \{x_1, x_2, x_3, x_4\}$, $E(G_1) = \{x_i x_{i+1} : 1 \leq i \leq 3\} \cup \{x_4 x_1\}$.

Note that G_1 is **isomorphic** to C_4 .

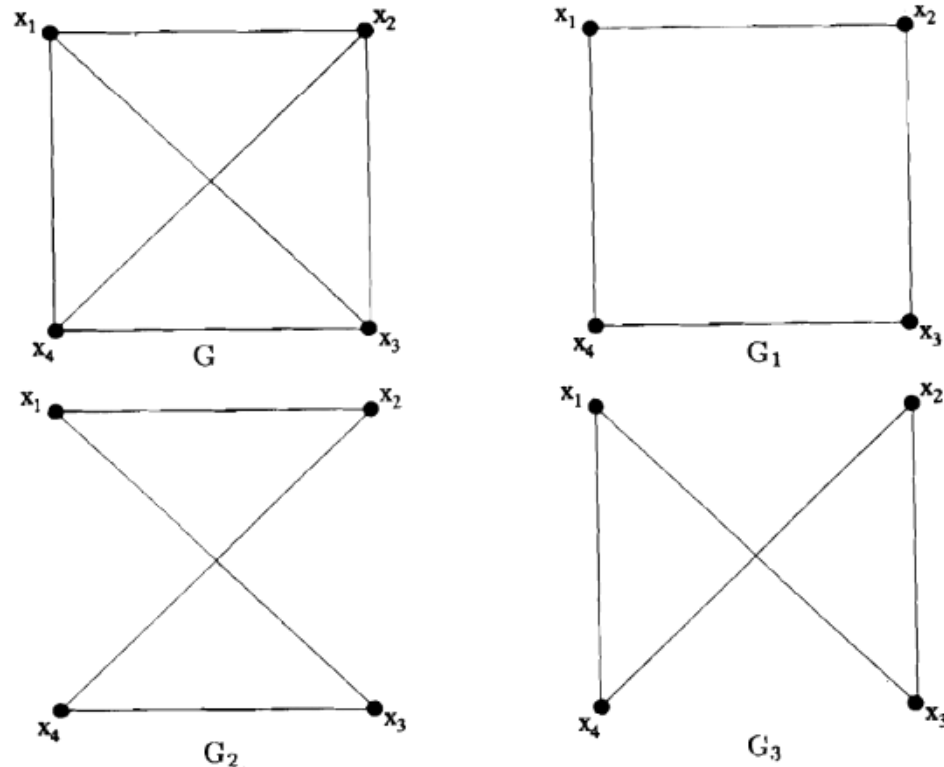
If $V(G_2) = \{x_8, x_9\}$, $E(G_2) = \{x_8 x_9\}$, then G_2 is **isomorphic** to K_2 .

Also the graph G_3 is **isomorphic** to C_7 , where

$V(G_3) = \{x_9, \dots, x_{15}\}$, $E(G_3) = \{x_{15} x_9\} \cup \{x_i x_{i+1} : 9 \leq i \leq 14\}$

Subgraphs(Contd....)

Example 12: Consider $G = K_4$ on four vertices x_1, x_2, x_3, x_4 as shown in Fig.25. From the figure you can see G_1, G_2, G_3 are subgraphs of G , with $V(G_1)=V(G_2)=V(G_3)=V(G)$. So, G_1, G_2 and G_3 are spanning subgraphs of the graph G .



Summary

- A simple graph G consists of a finite non-empty set V of points together with a prescribed set E of 2 element subsets of V .
- The complete graph K_n is a graph with n vertices such that every vertex is joined to every other vertex by an edge.
- A cycle is a circuit in which the only repeated vertex is the first vertex, which is the same as the last vertex.
- The number of edges incident with a vertex in a graph G gives the degree of the vertex. A graph having the same degree of all its vertices is regular.

Thank
you



V-LRN

VIRTUAL LEARNING NETWORK

Discrete Mathematics

Block – 4

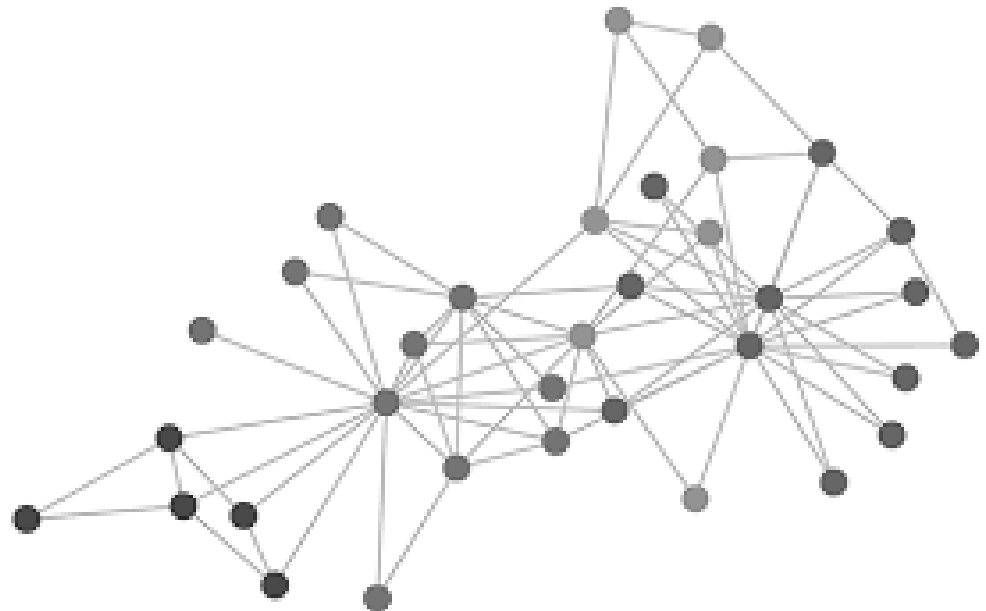
Unit - 2

Connectedness

Virtual Learning Network

Topics to be Covered

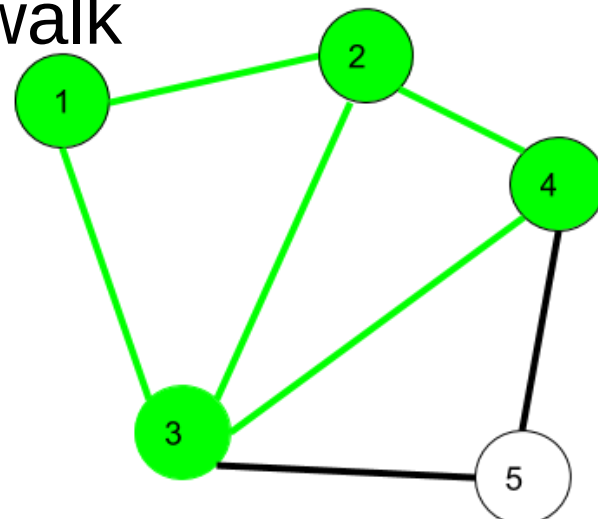
- Connected Graphs
 - Paths, Circuits and Cycles
 - Components
 - Connectivity
- Bipartite Graphs
- Trees



Connected Graphs

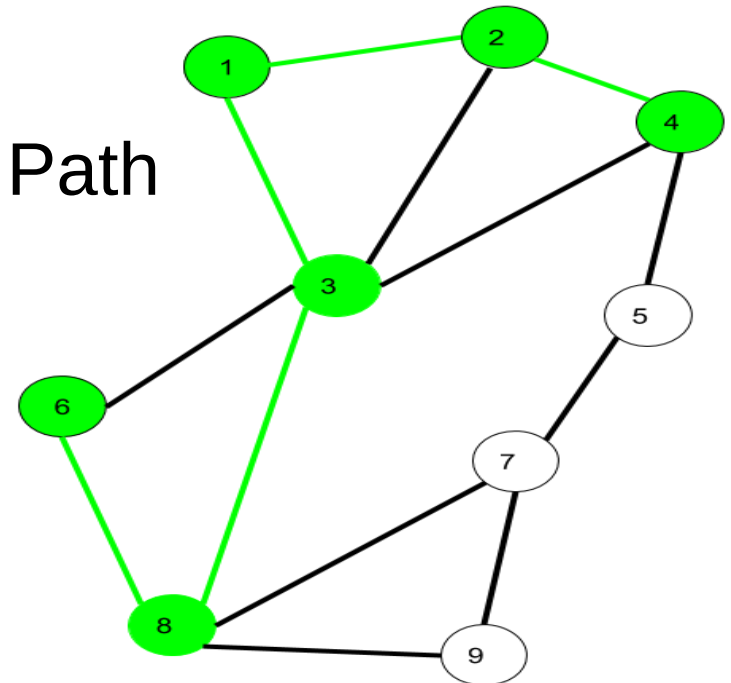
- **Paths, Circuits and Cycles**

- **Walk** : A walk is a sequence of vertices and edges of a graph i.e. if we traverse a graph then we get a walk.
- Vertex can be repeated
- Edges can be repeated
- Here $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 3$ is a walk
- Walk can be open or closed.
- Walk can repeat anything (edges or vertices).



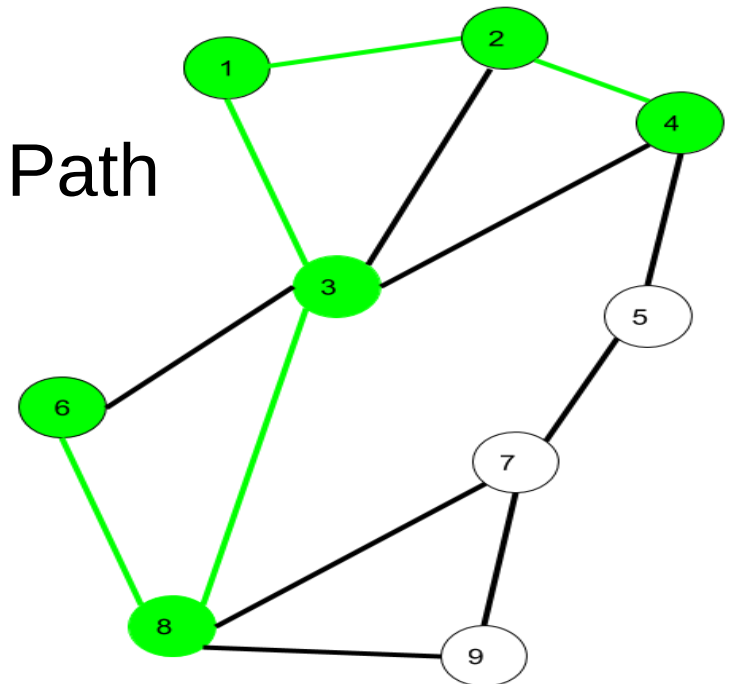
Connected Graphs(Contd....)

- **Path** : It is a trail in which neither vertices nor edges are repeated i.e. if we traverse a graph such that we do not repeat a vertex and nor we repeat an edge. As path is also a trail, thus it is also an open walk.
- Vertex not repeated
- Edge not repeated
- Here 6- $\rightarrow$ 8- $\rightarrow$ 3- $\rightarrow$ 1- $\rightarrow$ 2- $\rightarrow$ 4 is a Path



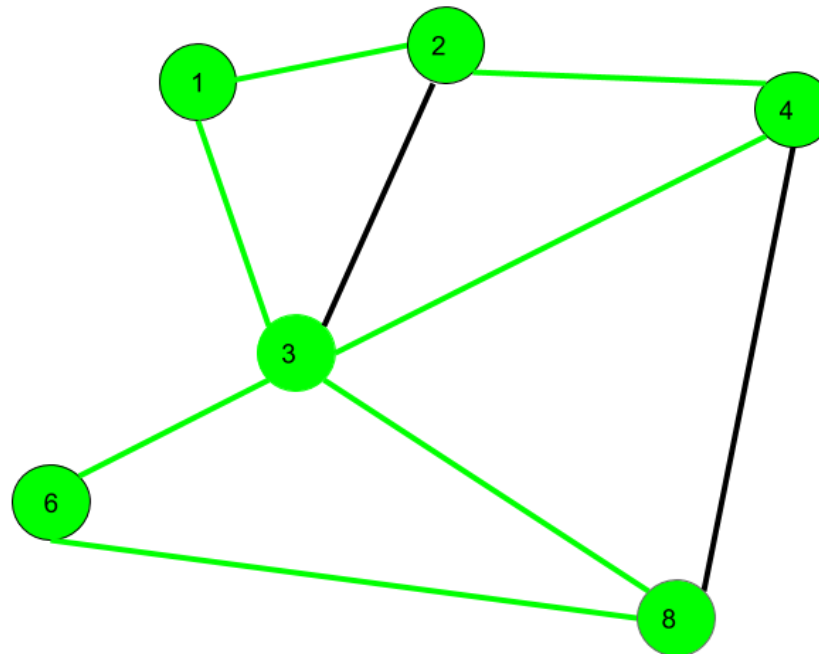
Connected Graphs(Contd....)

- **Path** : It is a trail in which neither vertices nor edges are repeated i.e. if we traverse a graph such that we do not repeat a vertex and nor we repeat an edge. As path is also a trail, thus it is also an open walk.
- Vertex not repeated
- Edge not repeated
- Here 6- $\rightarrow$ 8- $\rightarrow$ 3- $\rightarrow$ 1- $\rightarrow$ 2- $\rightarrow$ 4 is a Path



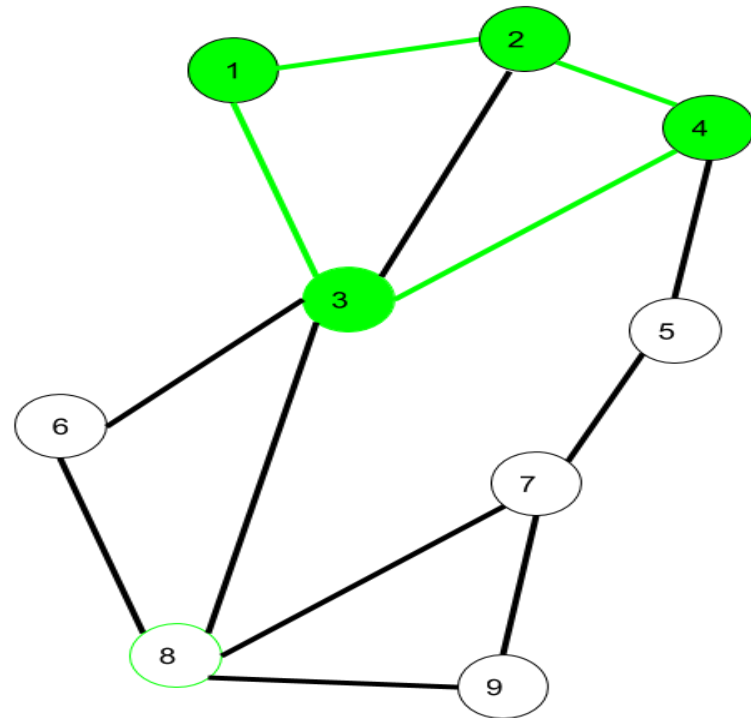
Connected Graphs(Contd....)

- **Circuit:** Traversing a graph such that not an edge is repeated but vertex can be repeated and it is closed also i.e. it is a closed trail.
- Vertex can be repeated
- Edge not repeated.



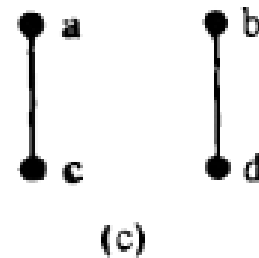
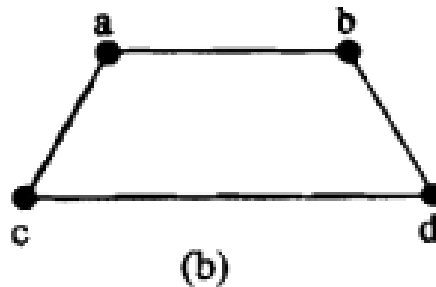
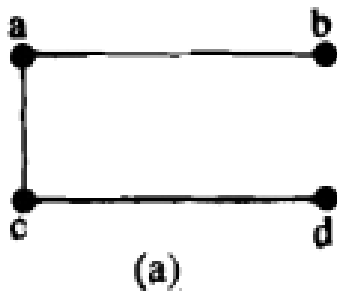
Connected Graphs(Contd....)

- **Cycle** : Traversing a graph such that we do not repeat a vertex nor we repeat a edge but the starting and ending vertex must be same i.e. we can repeat starting and ending vertex only then we get a cycle.
- Vertex not repeated
- Edge not repeated



Connected Graphs(Contd....)

- **Components:** A graph $G = (V, E)$ is called connected if for any two vertices $u, v \in V$, there exists a u - v walk in G . If G is not connected, then it is called disconnected.
- e, K_n is connected $\square n \geq 1$. However, a null graph, i.e., a graph whose edge set is empty, is totally disconnected.

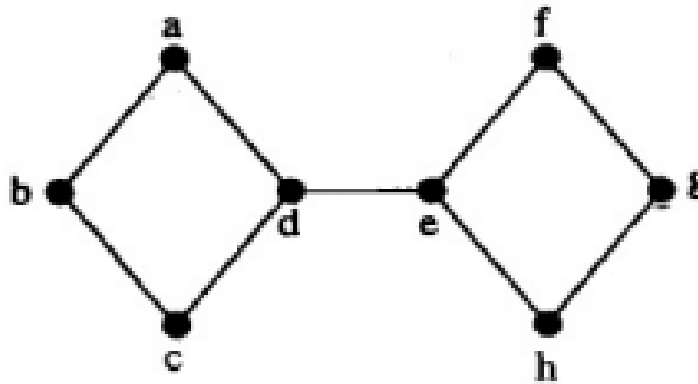


Connected Graphs(Contd....)

- Let $G = (V, E)$ be a graph. A subgraph H of G is called a component of G if H is connected and it is not a subgraph of any other connected subgraph of G . Thus, a component of G is, in a sense, a „maximal“ connected subgraph of G .
- **Theorem:** If G is a graph with n vertices and k components, then G can have at least $n-k$ edges, and at most $\frac{1}{2} * (n-k)(n-k+1)$ edges.
- **Corollary :** If G is a connected (n, m) -graph, then $n-1 \leq m \leq \frac{1}{2} * n(n-1)$.

Connected Graphs(Contd....)

- **Connectivity:** Removing the edge de does not mean that we remove the vertices d and e .



- An edge e of a connected graph G is called a bridge in G if the removal of e disconnects G . When we remove an edge e from the graph G , we denote the resulting graph by $G-e$.

Connected Graphs(Contd....)

- **Bipartite graph:**

- G is said to be bipartite if $V(G) = X \cup Y$, where X and Y are non-empty sets such that $X \cap Y = \phi$ and every edge in $E(G)$ has one end vertex in the set X and the other end vertex in the set Y . The sets X and Y form a partition of the set $V(G)$, and we often say that $X \cup Y$ is a bipartition of the graph G . We also denote such a graph by $G(X, Y)$.
- $V(C_6)$ can be partitioned into $\{a, c, e\}$ and $\{b, d, f\}$, C_6 is bipartite.

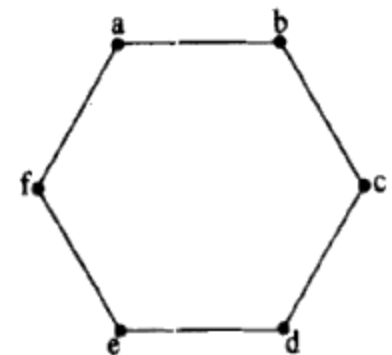
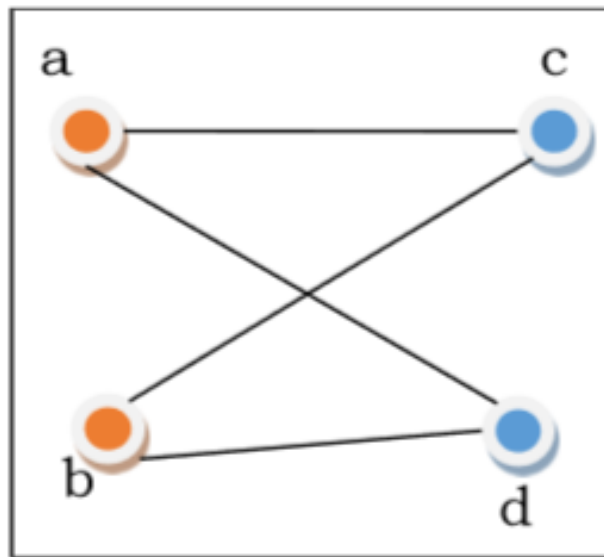


Fig. : C_6 is bipartite

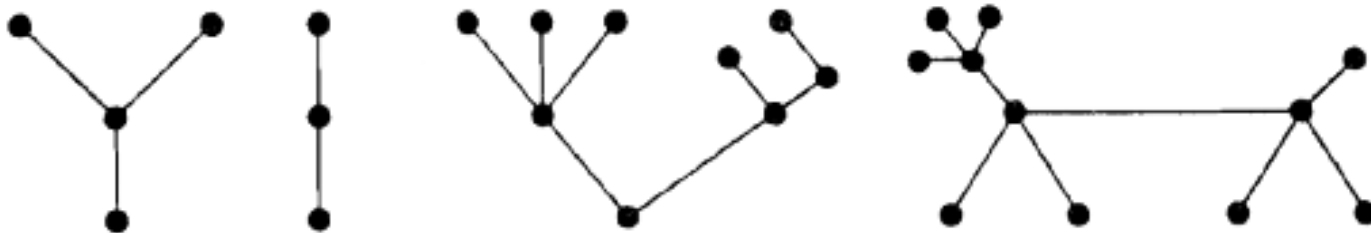
Connected Graphs(Contd....)

- **Complete Bipartite Graph** - A complete bipartite graph is a bipartite graph in which each vertex in the first set is joined to every single vertex in the second set. The complete bipartite graph is denoted by $K_{x,y}$ where the graph G contains x vertices in the first set and y vertices in the second set.



Connected Graphs(Contd....)

- A tree is a connected graph with no cycles.
- A forest is a graph, each of whose components is a tree.



Summary

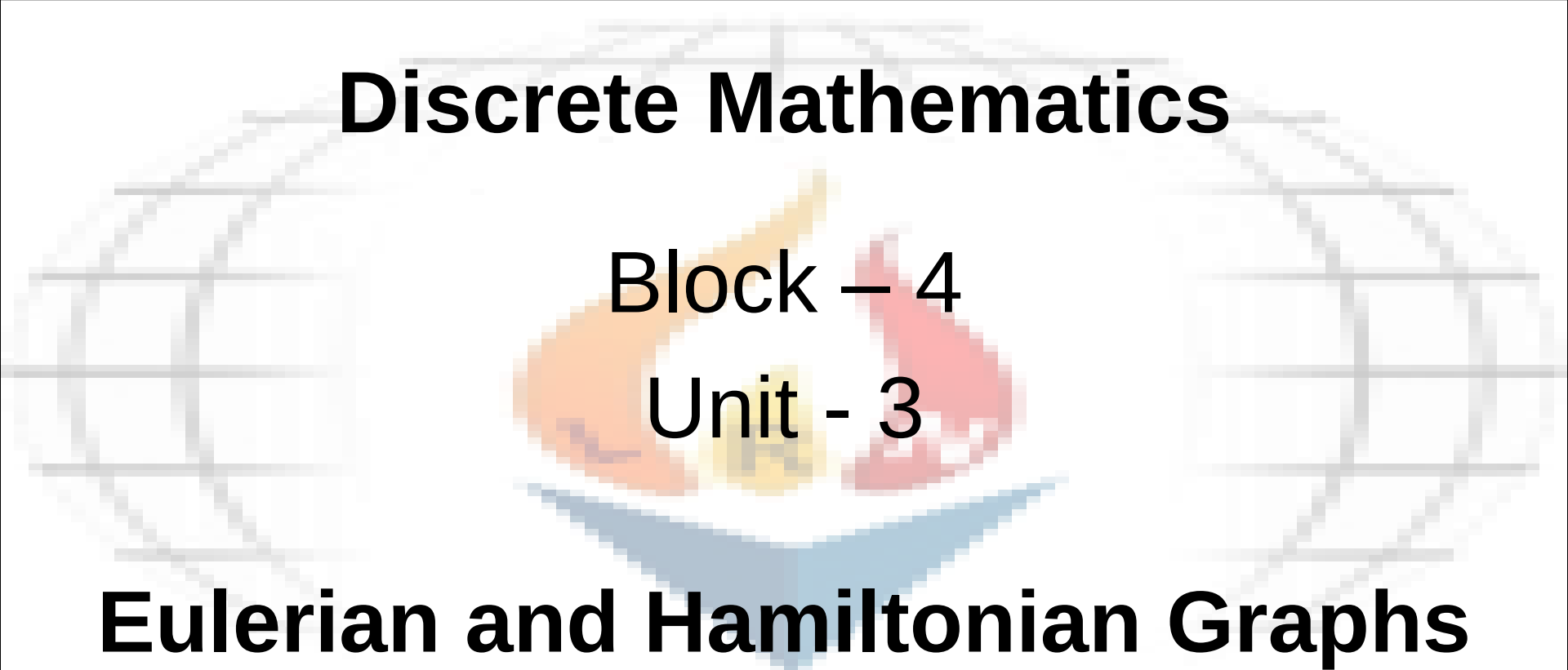
- In this unit we have discussed the following points.
 - The definition and use of the terms „walk“, „path“ and „cycle“ in a graph. We have also proved and used the fact that in every u - v walk there is a u - v path.
 - Properties of connected graphs, how to find components of a graph, the effect of removal of a vertex or an edge on the number $c(G)$ of the components of a graph G .
 - The definition and properties of bipartite graphs, and a characterisation of such graphs in terms of not containing odd cycles.

Thank
You



V-LRN

VIRTUAL LEARNING NETWORK



Discrete Mathematics

Block – 4

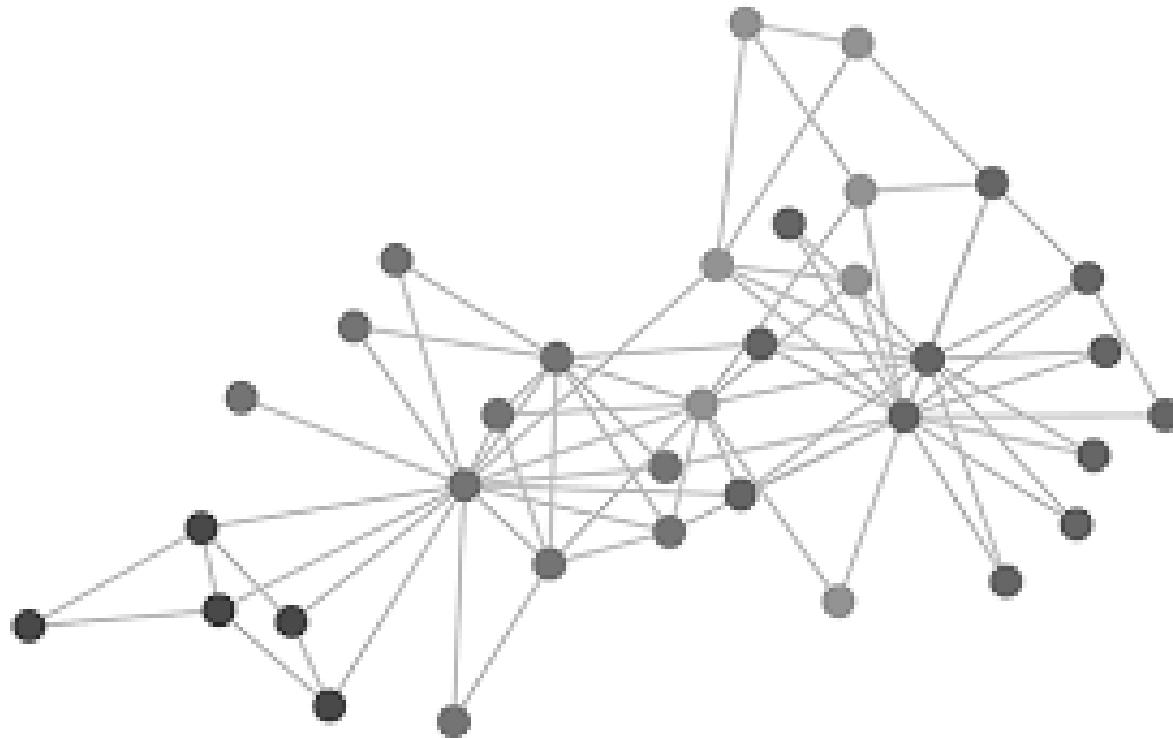
Unit - 3

Eulerian and Hamiltonian Graphs

Virtual Learning Network

Topics to be Covered

- Eulerian Graphs
- Hamiltonian Graphs
- Travelling Salesperson Problem



Introduction

- There is one more question similar to the Königsberg problem in recreational mathematics - which figures can be drawn without lifting the pen from the paper and without going over any of the lines twice? This question is also answered in A mathematical puzzle invented by Hamilton involves finding a cycle containing all the vertices of a certain graph.
- Leonhard Euler (1707-1783) 50 Graph Theory graph to contain a cycle containing all the vertices of the graph. Such as graph is called a Hamiltonian graph, in honour of Hamilton



Eulerian Graphs

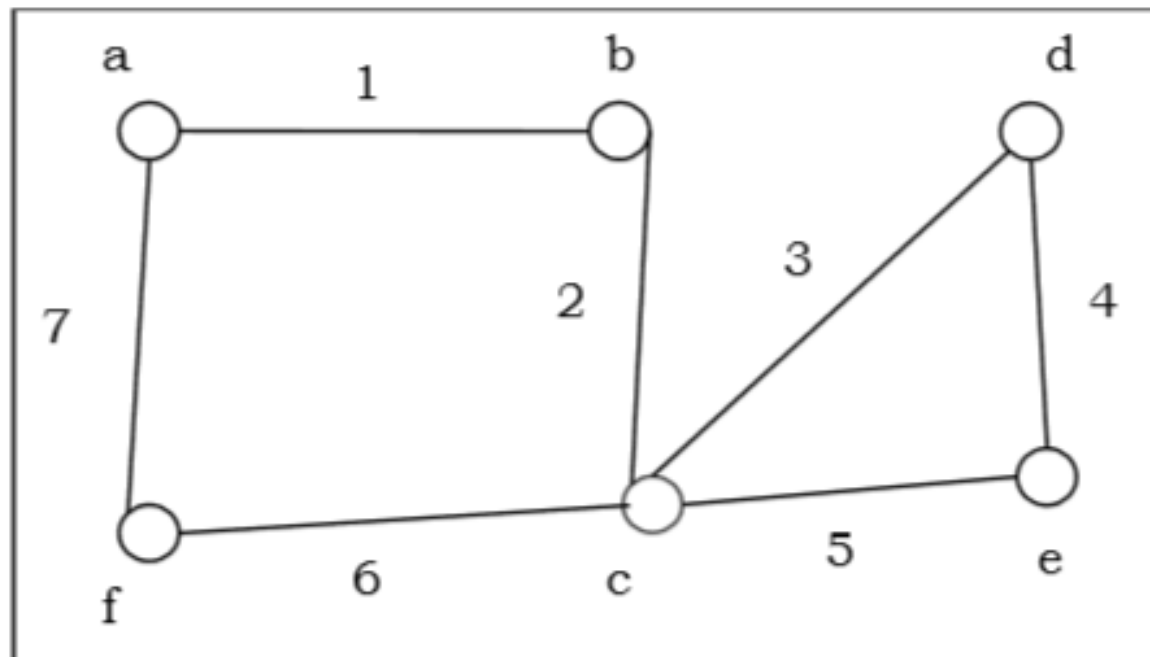
- **Euler Graph**
 - A connected graph G is called an Euler graph, if there is a closed trail which includes every edge of the graph G .
- **Euler Path**
 - An Euler path is a path that uses every edge of a graph exactly once. An Euler path starts and ends at different vertices.

Eulerian Graphs (Contd....)

- **Euler Circuit**

- An Euler circuit is a circuit that uses every edge of a graph exactly once.
- An Euler circuit always starts and ends at the same vertex.
- A connected graph G is an Euler graph if and only if all vertices of G are of even degree, and a connected graph G is Eulerian if and only if its edge set can be decomposed into cycles.

Eulerian Graphs (Contd....)

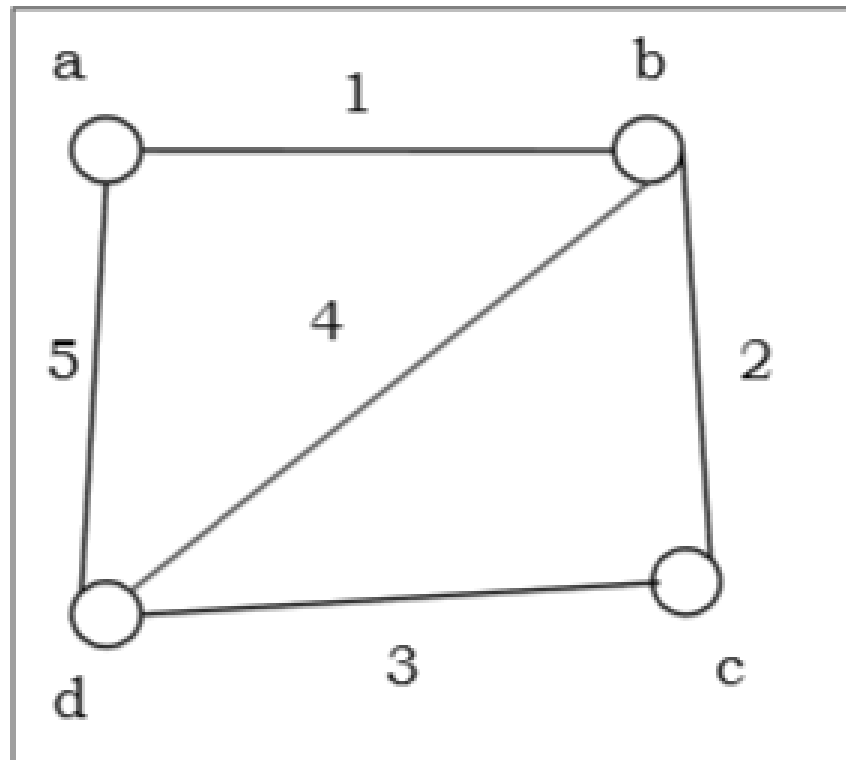


- The above graph is an Euler graph as a 1 b 2 c 3 d 4 e 5 c 6 f 7 g covers all the edges of the graph.

Eulerian Graphs (Contd....)

- **Non-Euler Graph**

- Here degree of vertex b and d is 3, an odd degree and violating the Euler graph condition.





Eulerian Graphs (Contd....)

1. A trail is a walk in which no edge is repeated.
2. A circuit is a trail whose starting vertex and end vertex are the same.
3. A trail which is not a circuit is sometimes called an open trail.
4. A circuit (resp. trail) in a graph G containing all the edges of G , is called an Eulerian circuit (resp. Eulerian trail).
5. A graph is Eulerian if it contains an Eulerian circuit.

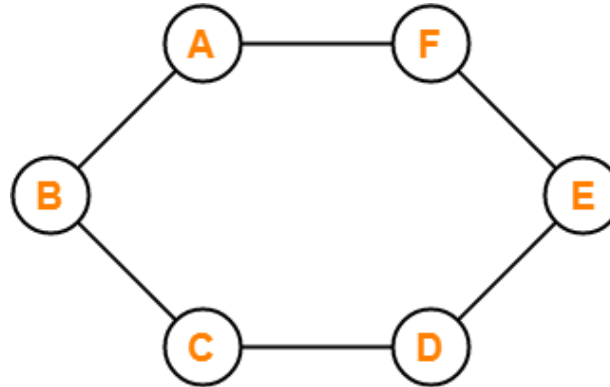


Hamiltonian Graph

- A Hamiltonian graph may be defined as-
 - If there exists a closed walk in the connected graph that visits every vertex of the graph exactly once (except starting vertex) without repeating the edges, then such a graph is called as a Hamiltonian graph.
 - Any connected graph that contains a Hamiltonian circuit is called as a Hamiltonian Graph.

Hamiltonian Graph(Contd....)

- The following graph is an example of a Hamiltonian graph



- This graph contains a closed walk ABCDEFA.
- It visits every vertex of the graph exactly once except starting vertex.
- The edges are not repeated during the walk.
- Therefore, it is a Hamiltonian graph.
- Alternatively, there exists a Hamiltonian circuit ABCDEFA in the above graph, therefore it is a Hamiltonian graph.



Travelling Salesperson Problem

- A travelling salesperson wants to visit a number of towns and return to her base.
- The travelling time between any two towns is known. How should she plan her journey so that she spends as short a time as possible but visits each town precisely once? This is known as the travelling salesperson problem.
- A weighted graph is a pair (G, f) , where G is a graph and f is a real valued function on the set $E(G)$.
- Let W be a walk in a weighted graph G . By the weight of the walk W , we mean the sum of the weights of all the edges in W .

Travelling Salesperson Problem

- Example: Consider the copy of a weighted K₆ given in Fig. Starting with the cycle {L, M, N, O, P, T, L}, modify it to a cycle of lesser weight. The number on the edge e indicates the weight $f(e)$ assigned to it.

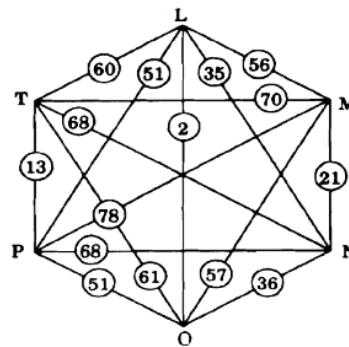
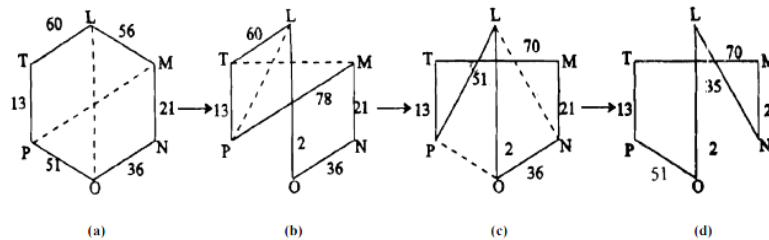


Fig.24



Summary

- In this unit we defined the following terms:
 - Eulerian circuit: A circuit in a graph is called Eulerian if each edge of the graph occurs exactly once in the circuit.
 - Eulerian graph: A connected graph is Eulerian if it contains an Eulerian circuit.
 - Open trail: A trail is open if the initial and end vertices of the trail are distinct.
 - Edge traceable graphs: A connected graph is edge traceable if it has an open trail.
 - Hamiltonian cycle: A cycle is Hamiltonian if each vertex of the graph occurs exactly once in the cycle.

Thank
You



V-LRN

VIRTUAL LEARNING NETWORK

Discrete Mathematics

Block – 4

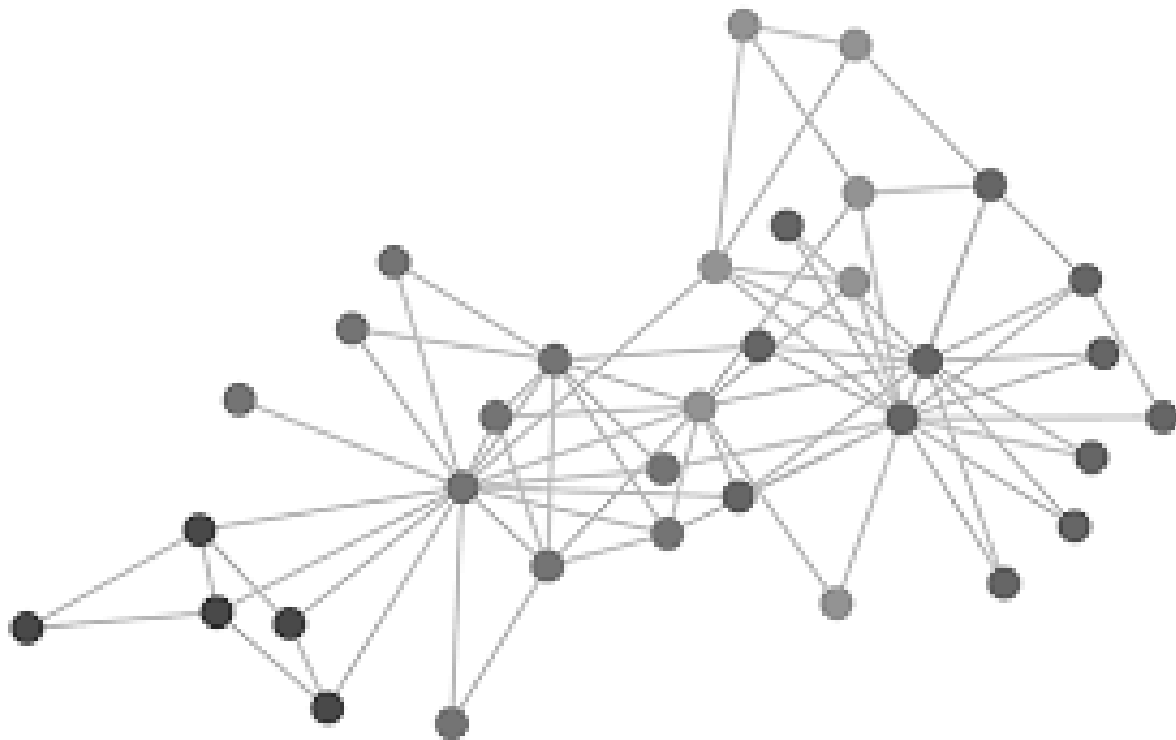
Unit - 4

Graph Colourings

Virtual Learning Network

Topics to be Covered

- Vertex Colouring
- Edge Colouring
- Planar Graphs
- Map Colouring Problem



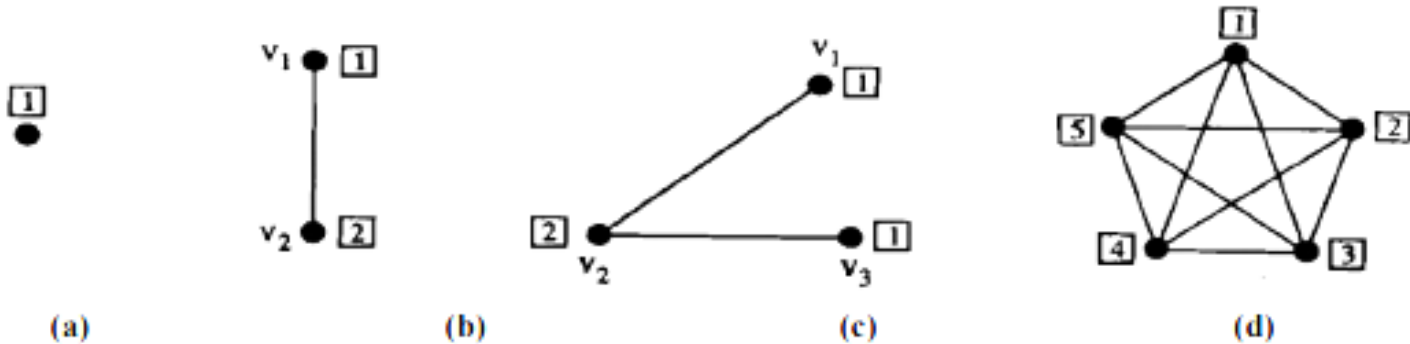
Introduction

- This problem of finding the minimum number of colours needed to colour a given map is called the map colouring problem.
- In the map colouring problem, we ask for the minimum number of colours needed to carry out such a colouring.
- The general vertex colouring problem, which asks for the minimum number of colours needed to colour the vertices of a given graph, not necessarily planar, is interesting in itself.
- Analogous to the colouring of vertices, is the colouring of edges.

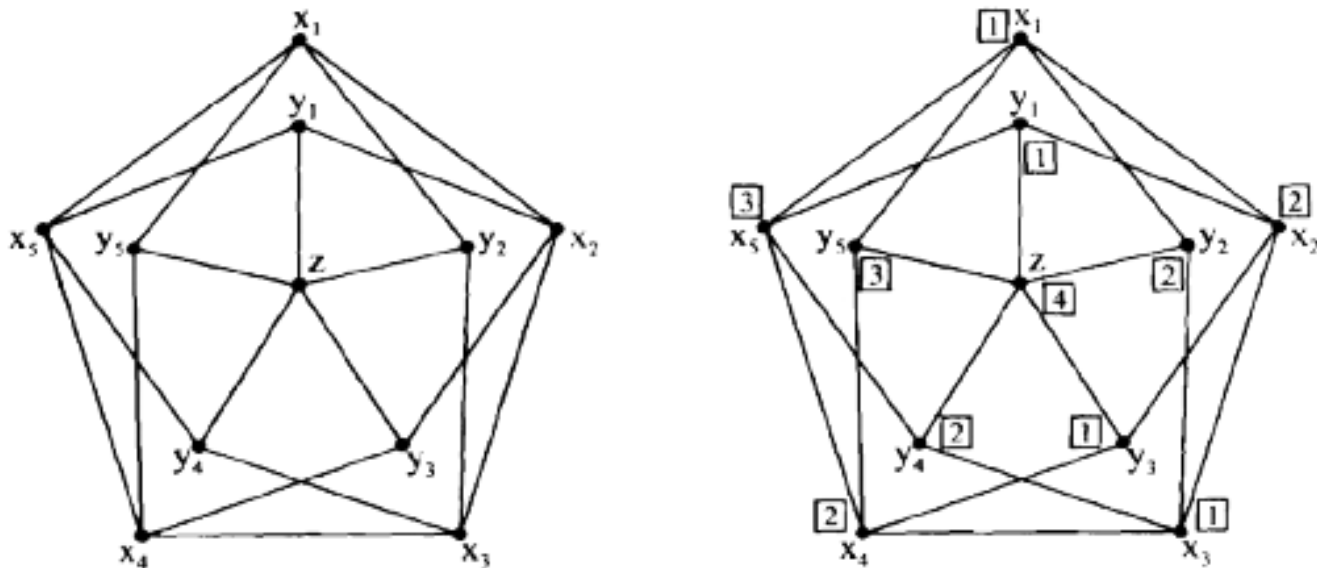
Vertex Colouring

- A vertex colouring of a graph G is an assignment of colours to vertices of G in such a way that no two adjacent vertices have the same colour.
- A graph is called k -vertex colourable if it has a vertex colouring of k colours.
- The minimum number of colours required to colour a graph G is called the vertex chromatic number of G , usually.
- Denoted as $\chi(G)$.

Vertex Colouring(Contd....)

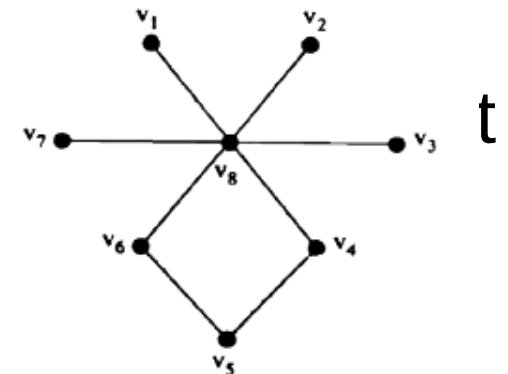


- Find the chromatic number of the Grötzsch graph



Vertex Colouring(Contd....)

- A subset S of the vertex set $V(G)$ of a graph G , is said to be an independent set if any two vertices in S are non-adjacent. An independent set is called maximal if it is not contained in any other independent set. The number of vertices in a largest independent set of G , is called the independence number of the graph G , and is denoted by $\alpha(G)$.
- Find three different maximal independent sets in the graph
 - The graph has the following maximal sets : $\{v_8, v_5\}$, $\{v_5, v_1, v_2, v_3, v_7\}$, $\{v_1, v_7\}$

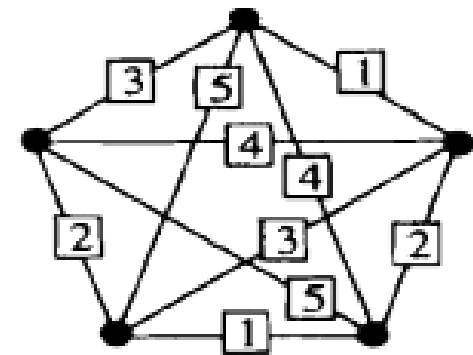
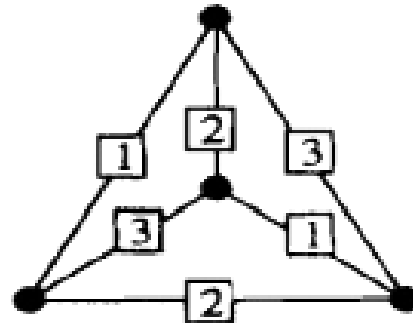
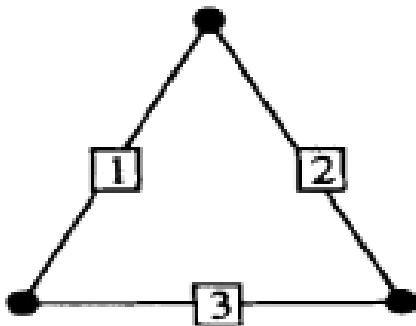


Edge Colouring

- A k -edge colouring of a graph G is an assignment of k colours to the edges of G in such a way that no two edges incident with the same vertex have the same colour.
- A graph is k -edge colourable if it has a k -edge colouring.
- The minimum number of colours required to colour the edges of a graph is called the edge chromatic number of G , usually denoted by $\chi'(G)$

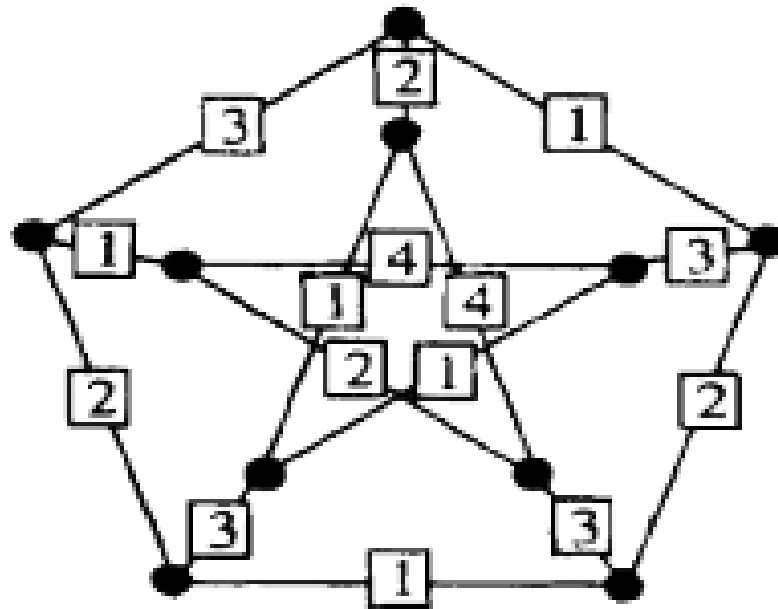
Edge Colouring(Contd....)

- Colour the edges of the graphs K_3 , K_4 , K_5 .
- **Solution** : The colouring of K_3 , K_4 , K_5 is given in Fig. 14. Here no two adjacent edges have received the same colour. In all the cases, we have used the least possible colours.



Edge Colouring(Contd....)

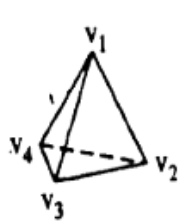
- Give an edge colouring of the Petersen graph.
- 4-edge colouring of the Petersen graph.



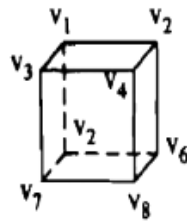
- Again no two adjacent edges have received the same colour. You can quickly check that three colours will not be enough. So $\chi'(G) = 4$.

Edge Colouring(Contd....)

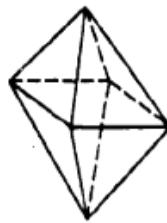
- Planar Graphs:** A graph G is called planar if it can be drawn on a plane in such a way that no two edges cross each other at any point except possibly at a common end vertex. Such a drawing is called a plane drawing.



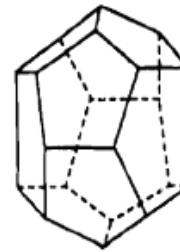
tetrahedron



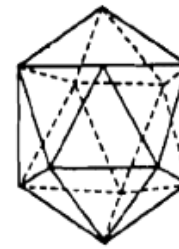
cube



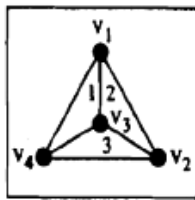
octahedron



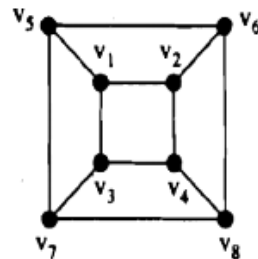
dodecahedron



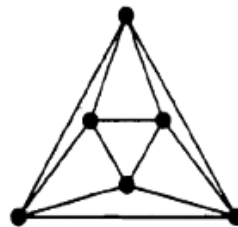
icosahedron



tetrahedron



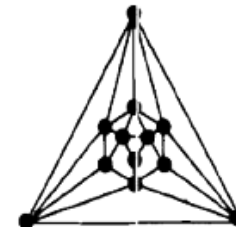
cube



octahedron



dodecahedron



icosahedron

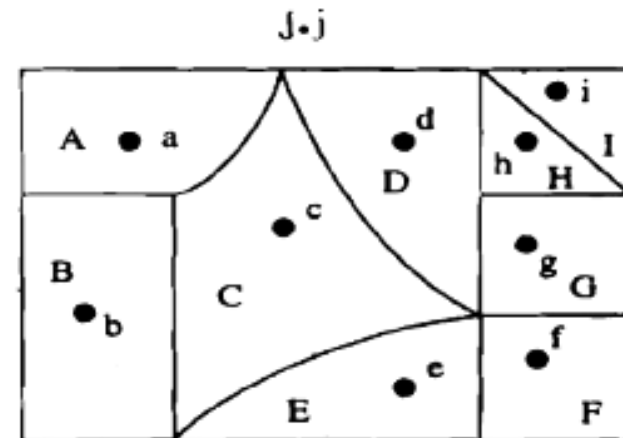
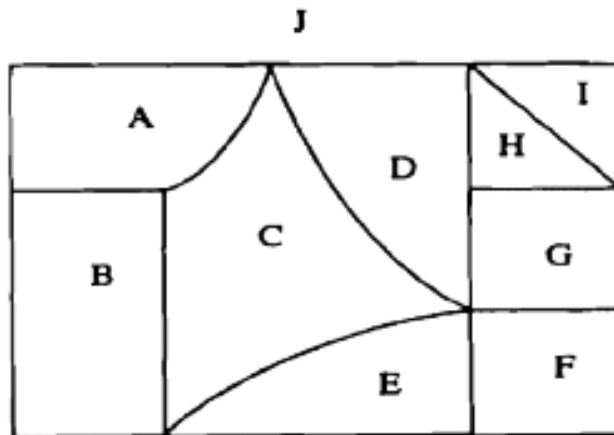
Edge Colouring(Contd....)

- Theorem (Euler's formula)** : If G is a connected planar (p,q) -graph, then the number r of the regions of G is given by $r = q - p + 2$.

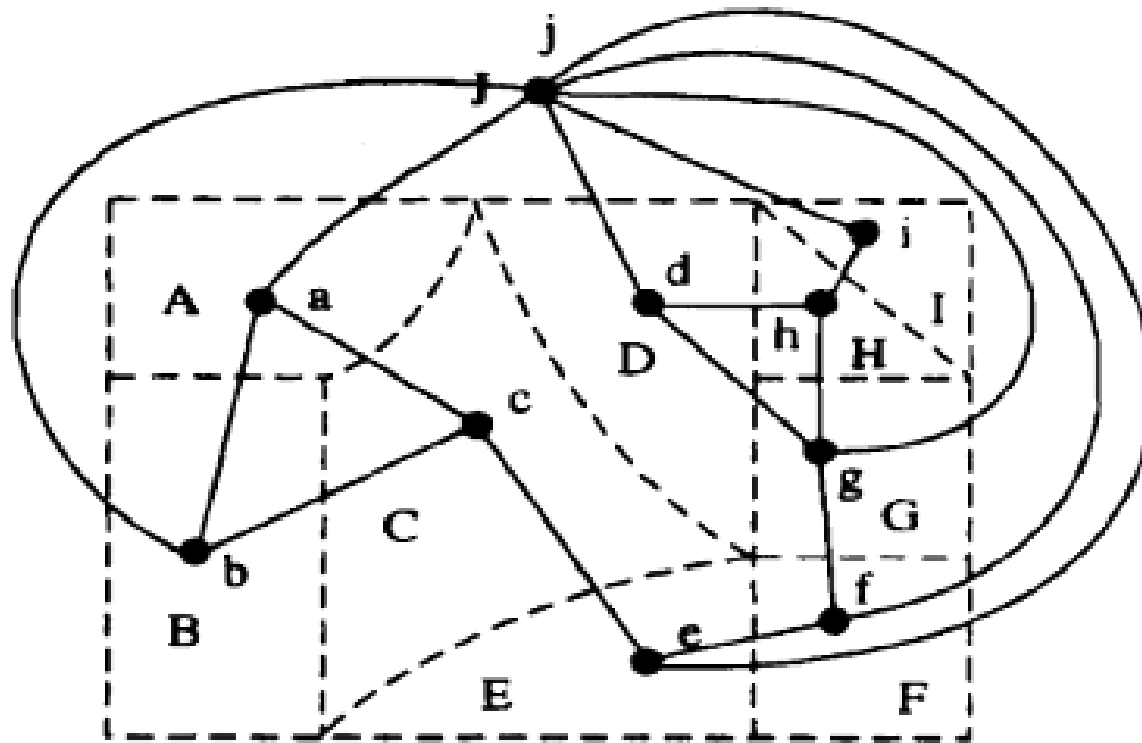
Graph	p	q	r	$p - q + r$
Fig.	6	7	3	2
Tetrahedron	4	6	4	2
Cube	8	12	6	2
Octahedron	6	12	8	2
Dodecahedron	20	30	12	2
Icosahedron	12	24	18	2

Map Colouring Problem

- **Theorem (Euler's formula)** : If G is a connected planar (p,q) -graph, then the number r of the regions of G is given by $r = q - p + 2$.

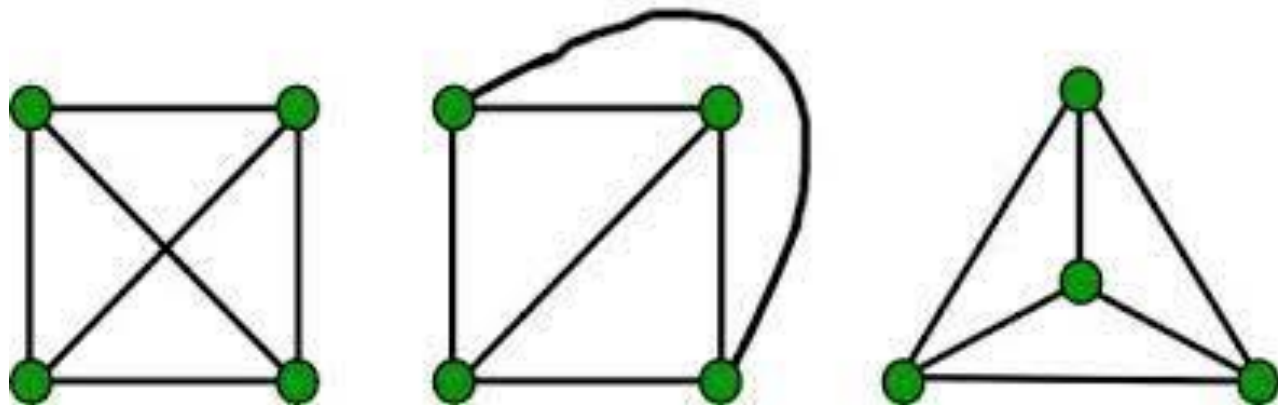


Map Colouring Problem



Summary

- Vertex colouring of a graph
- Vertex chromatic number of graph
- A colour class of a colouring
- Independent set
- Edge colouring of a graph
- Edge chromatic number of a graph
- Planar graph and Subdivision of a graph



Thank
you